



LAPORAN PROJECT WORK

PENGEMBANGAN APLIKASI PARKIR KENDARAAN MENGUNAKAN FRAMEWORK *CODEIGNITER* 4 DENGAN METODE *RESTFULL* API UNTUK MENINGKATKAN EFISIENSI PARKIR DI SMK PGRI 03 MALANG

NADIRA DWI SETIAWATI

22236/1944.065

GURU PEMBIMBING :

Ikrar Auliady, S.Kom

Yanuar Setyoningsih, S.Pd

BIDANG KEAHLIAN TEKNOLOGI INFORMASI DAN KOMUNIKASI

PROGRAM KEAHLIAN PENGEMBANGAN PERANGKAT LUNAK DAN GIM

KONSENTRASI KEAHLIAN REKAYASA PERANGKAT LUNAK

SMK PGRI 3 MALANG

2022

LEMBAR PERSETUJUAN

Judul : PENGEMBANGAN APLIKASI PARKIR KENDARAAN
MENGUNAKAN *FRAMEWORK CODEIGNITER 4* DENGAN
METODE *RESTFULL* API UNTUK MENINGKATKAN EFISIENSI
PARKIR DI SMK PGRI 03 MALANG
Oleh : NADIRA DWI SETIAWATI
NIS : 22236/1944.065

Mengetahui,

Pembimbing 1

Pembimbing 2

Ikrar Auliady, S.Kom

Yanuar Setyoningsih, S.Pd

Menyetujui,

Kepala Laboratorium TIK

Guru Wali

Eko Mulyanto, S.Pd

Soebariadi, S. Si

Kepala Bidang TIK

Yanri Nur Wibowo, ST

LEMBAR PENGESAHAN

Judul : PENGEMBANGAN APLIKASI PARKIR KENDARAAN
MENGUNAKAN *FRAMEWORK CODEIGNITER 4* DENGAN
METODE *RESTFULL* API UNTUK MENINGKATKAN EFISIENSI
PARKIR DI SMK PGRI 03 MALANG

Oleh : NADIRA DWI SETIAWATI
NIS : 22236/1944.065

Tanggal :
Ujian

Pembimbing :

Pembimbing 1

Pembimbing 2

Ikrar Auliady, S.Kom

Yanuar Setyoningsih, S.Pd

Penguji :

Penguji 1

Penguji 2

(.....)

(.....)

UCAPAN TERIMA KASIH

Pertama-tama penulis mengucapkan puji syukur dan terimakasih yang sedalam-dalamnya kepada Tuhan Yang Maha Esa yakni Allah s.w.t atas berkah kesehatan dan karunianya sehingga penulis dapat menyelesaikan penulisan laporan ini. Tidak lupa sholawat serta salam selalu penulis haturkan kepada junjungan besar Nabi Muhammad SAW.

Laporan ini disusun untuk memenuhi salah satu syarat untuk melaksanakan Prakerin (Praktek Kerja Industri) pada SMK PGRI 03 Malang. Dalam proses penyusunan laporan ini, penulis mendapatkan banyak sekali bantuan, bimbingan serta dukungan dari berbagai pihak. Untuk itu, rasa terima kasih penulis ucapkan kepada :

1. Tuhan Yang Maha Esa.
2. Kepada orang tua dan seluruh pihak yang telah memberikan masukan dan juga do'a yang selalu mengiringi penulis dalam pengerjaan Laporan Tugas Akhir ini sehingga dapat terselesaikan.
3. Bapak Lukman Hakim, ST selaku Kepala Sekolah SMK PGRI 03 Malang.
4. Bapak Yanri Nur Wibowo, ST selaku Kepala Bidang Teknik Informatika dan Komunikasi.
5. Bapak Soebariadi, S. Si selaku guru wali yang senantiasa memberi dukungan serta nasihat demi kemajuan anak didiknya dalam mengerjakan Laporan Tugas Akhir ini.
6. Bapak Ikrar Auliady, S.Kom dan Ibu Yanuar Setyoningsih, S.Pd selaku pembimbing yang telah menuntun penulis dalam pengerjaan Laporan Tugas Akhir ini dengan baik.
7. Teman-teman yang selalu memberikan semangat, do'a dan bimbingan (dalam bentuk bertukar pikiran dan pendapat) untuk pengerjaan Laporan Tugas Akhir ini.

KATA PENGANTAR

Pengumpulan Laporan Tugas Akhir tentang Pengembangan Aplikasi Parkir Kendaraan Menggunakan *Framework Codeigniter 4* Dengan Metode *Restfull* Api Untuk Meningkatkan Efisiensi Parkir Di Smk Pgri 03 Malang, dilatar belakangi oleh tuntutan penyerahan tugas akhir sebagai penentu keberangkatan kegiatan Prakerin (Praktek Kerja Industri). Pengembangan aplikasi ini akan digunakan untuk memudahkan pendataan parkir untuk siswa SMK PGRI 03 Malang.

Malang, 21 September 2022
Penulis

(Nadira Dwi Setiawati)
22236/1944.065

DAFTAR ISI

	Halaman
HALAMAN JUDUL	i
LEMBAR PERSETUJUAN	ii
LEMBAR PENGESAHAN	iii
UCAPAN TERIMA KASIH	iv
KATA PENGANTAR.....	v
DAFTAR ISI	vi
DAFTAR TABEL	ix
DAFTAR GAMBAR.....	x
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Perumusan Masalah	1
1.3 Tujuan dan Manfaat	1
1.4 Batasan Masalah	2
1.5 Metode Penelitian	2
1.6 Sistematika Penulisan	3
1.7 Rencana Kegiatan	4
BAB II TINJAUAN PUSTAKA	5
2.1 Aplikasi	5
2.2 <i>Framework</i>	5
2.3 HTML.....	6
2.4 CSS.....	7
2.5 JavaScript	8
2.6 PHP.....	10
2.7 <i>CodeIgniter</i>	11
2.8 Model View Controller (MVC).....	11
2.9 REST.....	13
2.10 API.....	13
2.11 <i>Restful API</i>	13
2.12 <i>Database</i>	15
2.13 DBMS	16

2.14 XAMPP	19
2.15 MySQL.....	20
2.16 Apache	24
2.17 CRUD.....	26
2.18 Bootstrap.....	28
2.19 Entity Relationship Diagram (ERD)	32
2.20 Data Flow Diagram (DFD)	35
2.21 <i>Flowchart</i>	38
2.22 <i>Visual Studio Code</i>	41
2.23 <i>Postman</i>	44
BAB III PROSES PERANCANGAN	47
3.1 Analisis Kebutuhan.....	47
3.1.1 Perangkat dan Alat	47
3.1.2 Software.....	47
3.1.3 Library	47
3.1.4 Data.....	47
3.2 Analisis Perancangan Sistem.....	48
3.2.1 Deskripsi Aplikasi Parkir	48
3.2.2 DFD.....	48
3.2.3 <i>Flowchart</i>	50
3.3 Rancangan <i>Database</i>	54
3.3.1 <i>Database</i>	54
3.3.2 ERD.....	55
3.3.3 Relasi Tabel	56
3.3.4 Tabel.....	56
3.4 Rancangan Tampilan Aplikasi.....	59
BAB IV HASIL DAN PEMBAHASAN	69
4.1 Hasil Pembuatan Aplikasi Parkir	69
4.2 Hasil Pembuatan Sistem <i>Database</i>	78
BAB V KESIMPULAN DAN SARAN	84
5.1 Kesimpulan	84
5.2 Saran	84
DAFTAR PUSTAKA	xiii

BIODATA PENULIS	xvi
-----------------------	-----

DAFTAR TABEL

Tabel

Tabel 1.1 Rencana Kegiatan	4
<i>Table 3.1 Siswa</i>	56
Tabel 3.2 Kendaraan	56
Tabel 3.3 Aktivitas	57
Tabel 3.4 <i>Oauth Access Token</i>	57
Tabel 3.5 <i>Oauth Clients</i>	57
Tabel 3.6 <i>Oauth Refresh Token</i>	58
Tabel 3.7 <i>Oauth Users</i>	58

DAFTAR GAMBAR

Gambar	Halaman
2.1 Aplikasi	5
2.2 <i>Framework</i>	5
2.3 HTML	6
2.4 CSS.....	7
2.5 JavaScript	8
2.6 PHP	10
2.7 <i>CodeIgniter</i>	11
2.8 MVC.....	11
2.9 <i>Restful API</i>	13
2.10 <i>Database</i>	15
2.11 DBMS.....	16
2.12 XAMPP.....	19
2.13 MySQL	20
2.14 Apache	24
2.15 CRUD	26
2.16 Bootstrap	28
2.17 Simbol Entitas	32
2.18 Simbol Atribut	33
2.19 Simbol Relasi	33
2.20 Simbol DFD.....	36
2.21 Simbol <i>Flowchart</i>	39
2.22 <i>Visual Studio Code</i>	41
2.23 <i>Postman</i>	44
3.1 DVD Level 0	48
3.2 DFD Level 1.....	48
3.3 DFD Level 2.....	49
3.4 <i>Flowchart Login Admin</i>	50
3.5 <i>Flowchart Login Satpam</i>	51
3.6 <i>Flowchart Halaman Admin</i>	52
3.7 <i>Flowchart Halaman Satpam</i>	53
3.8 <i>Database</i>	54

3.9 ERD	55
3.10 Relasi Tabel	56
3.11 <i>Home</i>	59
3.12 <i>About</i>	59
3.13 <i>Login</i>	60
3.14 <i>Dashboard Admin</i>	60
3.15 <i>Dashboard Satpam</i>	61
3.16 Aktivitas (Admin)	61
3.17 Aktivitas (Satpam)	62
3.18 Detail Aktivitas	62
3.19 Kelola Siswa	63
3.20 Tambah Siswa	63
3.21 Ubah Siswa	64
3.22 Kelola Kendaraan	64
3.23 Tambah Kendaraan	65
3.24 Ubah Kendaraan	65
3.25 Kelola Aktivitas (Admin)	66
3.26 Kelola Aktivitas (Satpam)	66
3.27 Tambah Aktivitas	67
3.28 Ubah Aktivitas	67
3.29 <i>Logout</i>	68
4.1 Halaman <i>Home</i>	69
4.2 Halaman <i>About</i>	69
4.3 Halaman <i>Login</i>	70
4.4 Halaman <i>Dashboard Admin</i>	70
4.5 Halaman <i>Dashboard Satpam</i>	71
4.6 Halaman Aktivitas (Admin)	71
4.7 Halaman Aktivitas (Satpam)	72
4.8 Halaman Detail Aktivitas	72
4.9 Halaman Kelola Siswa	73
4.10 Halaman Tambah Siswa	73
4.11 Halaman <i>Update</i> Siswa	74
4.12 Halaman Kelola Kendaraan	74

4.13 Halaman Tambah Kendaraan.....	75
4.14 Halaman <i>Update</i> Kendaraan	75
4.15 Halaman Kelola Aktivitas (Admin).....	76
4.16 Halaman Kelola Aktivitas (Satpam).....	76
4.17 Halaman Tambah Aktivitas	77
4.18 Halaman <i>Update</i> Aktivitas	77
4.19 Halaman <i>Logout</i>	78
4.20 <i>Database</i>	78
4.21 Tabel Siswa.....	79
4.22 Tabel Kendaraan	79
4.23 Tabel Aktivitas.....	79
4.24 Tabel <i>Oauth User</i>	80
4.25 Tabel <i>Oauth Client</i>	80
4.26 Tabel <i>Oauth Access Token</i>	81
4. 27 Tabel <i>Refresh Token</i>	81
4.28 Show Data	82
4.29 Create Data	82
4.30 <i>Update</i> Data	83
4.31 <i>Delete</i> Data.....	83

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada saat ini pandemi telah mereda sehingga berbagai macam kegiatan satu persatu mulai kembali seperti semula. Yang mulanya beroperasi secara daring (dalam jaringan), kembali menjadi kegiatan luring (luar jaringan). Salah satunya adalah kegiatan belajar mengajar yang telah dilaksanakan secara tatap muka pada sebagian besar sekolah yang ada di Indonesia, termasuk sekolah SMK PGRI 03 Malang telah melaksanakan pembelajaran secara luring 100% .

Siswa-siswi pada SMK PGRI 03 Malang, berasal dari berbagai daerah Baik dalam maupun luar Kota Malang. Melihat kondisi seperti itu, maka para guru di SMK PGRI 03 Malang memberikan kemudahan bagi siswa-siswinya dengan memperbolehkannya untuk membawa kendaraan pribadi untuk transportasi. Bukan hanya memperbolehkan membawa kendaraan, bahkan sekolah telah menyediakan fasilitas pendukungnya yaitu berupa tempat parkir yang luas di dalam lingkungan sekolah.

Namun, meski sekolah memberikan kemudahan dalam transportasi tetap saja SMK PGRI 03 Malang memiliki peraturan yang ketat dan sangat disiplin. Para siswa-siswi yang membawa kendaraan sendiri ke sekolah diharuskan untuk memperlihatkan STNK yang sesuai dengan kendaraan yang digunakan kepada satpam yang berjaga. Pada kesempatan ini, penulis akan mencoba untuk membuat aplikasi parkir kendaraan untuk SMK PGRI 03 Malang agar kegiatan pengecekan kelengkapan pengendara menjadi lebih praktis dan efisien.

1.2 Perumusan Masalah

Berdasarkan latar belakang yang telah dituliskan di atas, maka dapat dirumuskan beberapa masalah penting sebagai berikut.

1. Bagaimana cara membuat Aplikasi Parkir Kendaraan Menggunakan *Framework CodeIgniter 4* dengan Metode *Restfull* API untuk Meningkatkan Efisiensi Parkir di SMK PGRI 03 Malang?
2. Bagaimana cara mengoperasikan Aplikasi Parkir Kendaraan Menggunakan *Framework CodeIgniter 4* dengan Metode *Restfull* API untuk Meningkatkan Efisiensi Parkir di SMK PGRI 03 Malang?

1.3 Tujuan dan Manfaat

Tujuan umum dari proyek ini adalah untuk merancang dan mengembangkan aplikasi parkir kendaraan menggunakan *Framework CodeIgniter 4* dengan metode *Restfull API* untuk meningkatkan efisiensi parkir di SMK PGRI 03 Malang.

Tujuan umum tersebut kemudian dipecah menjadi tujuan khusus sebagai berikut:

1. Untuk memenuhi Tugas Akhir yang diberikan oleh sekolah sebagai syarat untuk melaksanakan PRAKERIN (Praktek Kerja Industri).
2. Meningkatkan tingkat efisiensi proses pendataan parkir pada SMK PGRI 03 Malang dengan menggunakan aplikasi parkir.
3. Untuk menguji program aplikasi yang dibuat dengan *Framework CodeIgniter 4* dengan metode *Restfull API*.

Manfaat dari dibuatnya Aplikasi Parkir Kendaraan Menggunakan *Framework CodeIgniter 4* dengan Metode *Restfull API* sebagai berikut :

1. Bagi siswa, hasil pengembangan ini diharapkan dapat meningkatkan kedisiplinan dan kelengkapan identitas para siswa SMK PGRI 03 Malang dalam berkendara.
2. Bagi guru, proyek ini dapat dikembangkan lagi dikemudian hari agar menjadi aplikasi yang lebih baik dan dapat diterapkan di SMK PGRI 03 Malang.
3. Bagi sekolah, hasil pengembangan aplikasi ini dapat memudahkan pendataan kendaran dalam area parkir sekolah, serta dapat mempersingkat waktu saat pengecekan kelengkapan.
4. Bagi peneliti, peneliti dapat memenuhi syarat untuk melaksanakan PRAKERIN (Praktek Kerja Industri).

1.4 Batasan Masalah

Untuk menghindari persepsi yang salah dan meluasnya pembahasan maka pembahasan masalah penelitian ini adalah :

1. Pembuatan proyek difokuskan hanya pada rest server saja.
2. Data yang digunakan adalah data yang ada pada SMK PGRI 03 Malang.
3. Dalam pembuatan aplikasi ini menggunakan software *Visual Studio Code*.
4. *Database* yang digunakan adalah MySQL.
5. Aplikasi ini menggunakan *Framework CodeIgniter 4*.

1.5 Metode Penelitian

a. Proses Pengumpulan Data

Penulis mengumpulkan data dengan mencari Informasi yang ada tentang parkir kendaraan melalui Google .

Berikut data yang penulis cari :

1. Data mengenai rancangan sistem dan *database* parkir kendaraan.
2. Data tentang apa yang dibutuhkan dalam membuat aplikasi parkir kendaraan.
3. Data mengenai tampilan *Website* yang nyaman dilihat dan mudah dipahami.

b. Alat dan Bahan

Alat dan Bahan untuk mengerjakan proyek aplikasi parkir kendaraan adalah sebagai berikut :

1. Laptop/smartphone
2. Koneksi Internet
3. Alat Tulis

c. Teknik Analisis Penelitian

Penulis melakukan penelitian secara tidak langsung dengan cara melihat cara kerja Aplikasi parkir kendaraan pada media pencarian di laptop/smartphone.

Penulis memanfaatkan data yang sudah dikumpulkan sebagai refrensi pengerjaan project dan menyusunnya pada BAB II dan III. Mulai dari rancangan sistem, *database* maupun tampilan *Website*.

1.6 Sistematika Penulisan

Dalam laporan ini, penulis membagi menjadi 5 bab. Adapun sistematika penulisan Laporan Tugas Akhir adalah sebagai berikut.

BAB I PENDAHULUAN

Pendahuluan berisi tentang latar belakang masalah, rumusan masalah, batasan masalah, tujuan penelitian, dan manfaat penelitian.

BAB II

TINJAUAN PUSTAKA

2.1 Aplikasi



Gambar 2.1 Aplikasi

(Sumber : <http://asset-a.grid.id/crop/0x0:0x0/x/photo/2019/05/20/2108393760.jpg>)

Aplikasi merupakan sebuah perangkat lunak yang digabungkan penggunaannya dengan perangkat keras yang akan menjalankan perintah atau intruksi dari penggunaanya dalam mengolah kata, mengolah angka, dan lain sebagainya.

Menurut Jogiyanto : 1999, Aplikasi adalah suatu intruksi atau pernyataan yang terdapat pada suatu perangkat keras baik komputer ataupun *smartphone* yang di buat sedemikian rupa agar dapat mengolah sebuah masukan (*input*) menjadi keluaran (*output*).

2.2 Framework



Gambar 2.2 Framework

(Sumber : <https://bhintara.com/wp-content/uploads/2019/05/Jenis-Jenis-Framework.png>)

Framework adalah sebuah kerangka kerja yang digunakan untuk mengembangkan *website*. *Framework* ini diciptakan untuk membantu web *developer* dalam menulis baris kode. Dengan menggunakan *Framework* penulisan kode akan jauh lebih mudah, cepat, dan terstruktur rapi.

Framework memiliki fungsi utama untuk memudahkan web *developer* dalam membuat sebuah *Website*. Selain itu, *Framework* juga memiliki fungsi lain. Berikut di antaranya:

- a. Membuat kode program menjadi lebih terstruktur
- b. Meningkatkan keamanan
- c. Mempercepat pembuatan *Website*
- d. Pemeliharaan dan perawatan *Website* lebih mudah

Ada beragam jenis *Framework* yang sering digunakan untuk membuat *Website*. Setiap *Framework* memiliki fungsinya masing-masing dan juga menggunakan bahasa pemrograman yang berbeda juga. Nah, berikut ini adalah jenis-jenis *Framework* untuk membuat suatu *Website*.

- a. *Framework* CSS
 - 1) Bootstrap
 - 2) Semantic UI
 - 3) Materialize
- b. *Framework* JavaScript
 - 1) AngularJS
 - 2) ReactJS
- c. *Framework* PHP
 - 1) Symfony
 - 2) Laravel
 - 3) CodeIgniter

2.3 HTML



Gambar 2.3 HTML

(Sumber : https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcSZrv5FhnG8vBIDg_VEGdoWYxCsHcqOPOLiqQ&usqp=CAU)

Dikutip dari Wikipedia *Hyper Text Markup Language (HTML)* adalah sebuah bahasa markah yang digunakan untuk membuat sebuah halaman web, menampilkan berbagai informasi di dalam sebuah penjelajah web Internet dan pemformatan *hiperteks* sederhana yang ditulis dalam berkas format ASCII agar dapat menghasilkan tampilan wujud yang terintegrasi.

Menurut Suyanto (2007:83) "HTML itu adalah bahasa yang digunakan untuk menulis halaman web, biasanya menggunakan *extensi* .htm, .html atau .shtml".

HTML berawal dari bahasa *SGML (Standard Generalized Markup Language)* yang penulisannya disederhanakan. HTML dapat dibaca oleh berbagai macam platform. HTML juga merupakan bahasa pemrograman yang fleksible dan dapat digabungkan dengan bahasa pemrograman lain seperti PHP, ASP, JSP, JavaScript.

Beberapa tag dalam dokumen-dokumen HTML menentukan bagaimana teks diformat. Tag-tag yang lain memberitahukan komputer bagaimana menanggapi aksi-aksi yang datang dari pengguna. Kemudian tag lain yang penting adalah link yang mengandung *Uniform Resource Locator (URL)*, yang merujuk pada dokumen lain di *server* yang sama atau komputer lain yang ada di global jaringan internet.

2.4 CSS



Gambar 2.4 CSS

(Sumber : <https://srv1.portal.p-cd.net/850p/2019/07/22/175075-1563766195-790533.jpg>)

CSS adalah singkatan dari *Cascading Style Sheet* yang merupakan kumpulan perintah yang dibentuk dari berbagai sumber yang disusun menurut urutan tertentu sehingga mampu mengatasi konflik *style*. Pengertian CSS adalah salah satu bahasa desain web yang mengatur komponen dalam suatu web supaya web lebih terstruktur dan lebih seragam. CSS seperti halnya *styles* pada aplikasi pengolahan kata Microsoft Word yang bisa mengatur beberapa *style*, misalnya subbab, *heading*, *bodytext*, *footer*, *images*, dan *style* lainnya untuk dapat dipakai bersama-sama dalam beberapa file. CSS biasanya digunakan untuk memformat tampilan halaman web yang dibuat dengan bahasa HTML dan XHTML.

Jayan (2010:2), menyatakan bahwa CSS adalah singkatan dari *Cascading Style Sheet*. Kegunaan CSS yaitu untuk mengatur tampilan dokumen HTML, sebagai contoh pengaturan jarak antar baris, teks, format border, warna bahkan hingga penampilan file gambar.

Fungsi CSS pada web yaitu untuk merancang, mendesain, merubah dan membentuk halaman sebuah web atau blog dan isi dari halaman web tersebut yaitu tag-tag html. Logikanya fungsi CSS dalam html yaitu mengubah tag-tag html yang sederhana menjadi lebih menarik dan fungsional. Jadi fungsi CSS pada html sangatlah penting.

2.5 JavaScript



Gambar 2.5 JavaScript

(Sumber : https://gmedia.net.id/upload/foto_artikel/20200831OqzWyXZKch.png)

Menurut Mac Bride (2007) dalam bukunya berjudul, Java Script, disebutkan bahwa JavaScript adalah bahasa pemrograman berbasis *browser*, dimana seluruh kodenya ditulis langsung ke dalam HTML dari halaman-halaman web yang kemudian diterjemahkan dan dieksekusi sebagai respon dari seluruh aktivitas yang terjadi pada halaman web.

Berbeda dengan HTML, CSS, dan PHP yang bahasa pemrogramannya diproses di balik *server*, dimana seluruh prosesnya akan menjadi tugas dan tanggung jawab seorang *back-end programmer*. Proses pemindaian bahasa pemrograman HTML dan CSS ini akan dilakukan oleh *web server*.

Sedangkan di JavaScript, bahasa script tersebut akan di-embed langsung ke halaman *website*. Itulah salah satu alasan mengapa JavaScript sangat dekat dengan seorang *Front End Developer* karena seluruh scriptnya akan langsung terunduh di perangkat yang digunakan dan tidak diproses melalui *server* lagi.

Digunakan oleh hampir 95% *Website* di dunia, tentu bukan hal yang sulit untuk membahas beberapa keunggulan menggunakan bahasa pemrograman JavaScript, antara lain yaitu:

- a. Dengan menggunakan JavaScript, web *browser* akan dapat langsung menginterpretasikan *script* yang telah dibuat secara otomatis melalui HTML.
- b. Bahasa pemrograman JavaScript mudah dipahami oleh para web *developer*, termasuk *fresh graduate* atau orang yang baru berkecimpung di dunia web *developer*.
- c. Penanganan *error* menjadi lebih mudah, sehingga ketika terjadi *error*, web *developer* dapat langsung menemukan dan memperbaikinya.
- d. *Multi Platform*, karena dapat digunakan di semua jenis sistem operasi komputer, seperti Windows, Linux, Android, dan beragam sistem operasi lainnya.
- e. JavaScript mudah dikembangkan karena mengusung konsep *multi platform* dan dinamis sehingga dapat dengan mudah untuk diperbaharui dan dikembangkan sesuai dengan kebutuhan *user* serta perkembangan teknologi setiap waktu.
- f. Bahasa pemrograman JavaScript sangat cepat, ringan dan *user friendly* karena tidak membutuhkan *resource memory* yang banyak. Hal ini tentu membuat para *developer* untuk mengembangkan berbagai aplikasi sesuai dengan kebutuhan *user*.

Meski mempunyai banyak keunggulan, layaknya sebuah sistem, bahasa pemrograman JavaScript ternyata juga mempunyai beberapa kekurangan yang perlu Anda ketahui, antara lain yaitu:

- a. Dengan sistemnya yang mudah dikembangkan dan mudah dipahami oleh para web *developer*, membuat bahasa pemrograman JavaScript mudah sekali untuk di *eksploitasi*, terutama bagi para web *developer* nakal.
- b. Sangat berisiko digunakan untuk kepentingan yang buruk, seperti penggunaan *Website* untuk keperluan *spy* atau memata-matai dan berbagai kejahatan *cyber* lainnya.
- c. Mempunyai banyak JavaScript *code snippet* sehingga membuatnya tidak dapat terenkripsi dengan baik atau malah cenderung tidak terenkripsi sama sekali.

- d. Tidak konsisten karena data dari masing-masing perangkat dirender berbeda satu sama lain.

2.6 PHP



Gambar 2.6 PHP

(Sumber : <https://badoystudio.com/wp-content/uploads/2019/08/apa-itu-php.png>)

Betha sidik dalam bukunya dengan judul Pemrograman Web Dengan PHP (2014:4), menyebutkan bahwa PHP sebagai bahasa pemrograman membuat dokumen HTML yang akan dieksekusi pada *server* web, dokumen yang dihasilkan bukanlah dokumen yang dibuat dengan menggunakan editor teks ataupun editor teks HTML.

Sibero (2011d:49) juga mendefinisikan PHP dengan arti yang atidak jauh berbeda, yang mana bahwa PHP sebagai bahasa yang memiliki hak cipta yang dikenal dengan *open source*, yang mana pengguna data dapat mengembangkan kode-kode pemrograman seusai dengan fungsi dan kebutuhannya.

Dari beberapa pengertian mengenai PHP menurut para ahli di atas dapat ditarik kesimpulan, bahwa PHP merupakan sebuah bahasa pemrograman yang tergabung dengan HTML, yang mana akan melakukan eksekusi pada server web. PHP ini memiliki hak cipta terbuka sehingga pengguna dapat mengembangkan kode-kode fungsi sesuai dengan kebutuhan yang diinginkan.

Adapaun fungsi PHP, yaitu:

- a. Mempersingkat tatanan HTML dan CSS
- b. *Input* data
- c. Manajemen cookie dan session
- d. Kompres teks

2.7 CodeIgniter



Gambar 2.7 CodeIgniter

(Sumber : <https://codesyariah122.github.io/assets/images/post/codeigniter.png>)

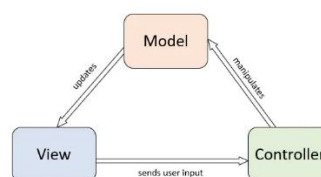
CodeIgniter adalah sebuah *web application network* yang bersifat *open source* yang digunakan untuk membangun aplikasi php dinamis. CodeIgniter menjadi sebuah *Framework* PHP dengan model MVC (*Model*, *View* dan *Control*) untuk membangun *Website* dinamis dengan menggunakan PHP yang dapat mempercepat pengembangan untuk membuat sebuah aplikasi web.

Menurut Ridha (2007:2) “CodeIgniter adalah sebuah *Framework* PHP yang bersifat *Open Source* dan dapat digunakan untuk mempercepat pengembangan dalam membuat aplikasi web”.

Keunggulan *Framework* CodeIgniter :

- a. Bersifat *free* dan *open-source*.
- b. Memiliki ukuran yang kecil dibandingkan dengan *Framework* lain. Setelah proses *instalasi*, *Framework* CodeIgniter hanya berukuran kurang lebih 2MB (tanpa dokumentasi atau jika direktori *user_guide* dihapus). Dokumentasi CodeIgniter memiliki ukuran sekitar 6MB.
- c. Aplikasi yang dihasilkan bisa berjalan cepat.
- d. Menggunakan pola desain MVC sehingga satu file tidak terlalu berisi banyak kode. Hal ini menjadikan kode lebih mudah dibaca, dipahami, dan dipelihara di kemudian hari.
- e. Dapat diperluas sesuai dengan kebutuhan.

2.8 Model View Controller (MVC)



Gambar 2.8 MVC

(Sumber : <https://bmu-verlag.de/wp-content/uploads/image1-6.png>)

Model View Controller merupakan suatu konsep dalam pembangunan aplikasi web. Terdapat 3 jenis komponen yang membangun suatu mvc dalam aplikasi yaitu:

- a. *Model*, merupakan bagian yang berhubungan dengan *database* untuk memanipulasi data seperti *query*, *insert*, *update*, dan *search*.
- b. *View* merupakan bagian yang menangani tampilan *interface* halaman web seperti kontak tels, *dropdown*, dll.
- c. *Controller*, bisa di katakan sebagai relasi hubungan antara bagian model dengan bagian *view*, dimana *controller* berfungsi menerima *request* dan data dari *user* kemudian menentukan apa yang akan di proses aplikasi.

Menurut Badiyanto (2013:49), MVC adalah *model* pembuatan program yang menerapkan arsitektur aplikasi menjadi tiga bagian memisahkan antara tampilan proses, tampilan dan bagian yang menghubungkan dengan *database* dan memisahkan proses bisnis dari pertimbangan antarmuka *user* agar para pengembang bisa lebih mudah mengembangkan salah satu bagian dari aplikasi sehingga tidak mempengaruhi bagian yang lain.

Dengan menggunakan Prinsip MVC, suatu aplkasi dapat dikembangkan secara terpisah antara *layer application logic* dan *presentation*. Ada beberapa manfaat yang didapatkan dalam pembangunan sistem dengan konsep MVC tersebut . Berikut ini merupakan beberapa manfaat penggunaan MVC.

- a. Komponen-komponen yang digunakan pada antarmuka dapat digunakan kembali pada sistem lain (*user interface reusable component*).
- b. Pengembangan aplikasi dengan antarmuka pengguna secara terpisah.
- c. Memiliki kemampuan pewarisan (*inheritance*) dari berbagai bagian yang berbeda pada suatu hirarki kelas.
- d. Kemudahan dalam mengatur tampilan (*control style*) yang menyediakan fitur umum secara terpisah dengan fitur yang akan ditampilkan oleh aplikasi yang dikembangkan.

2.9 REST

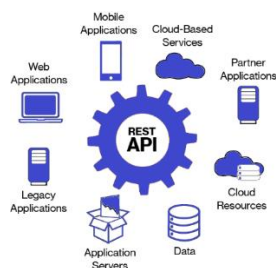
Representational State Transfer (REST) adalah arsitektur perangkat lunak yang memberlakukan syarat mengenai cara API bekerja. REST pada awalnya dibuat sebagai panduan untuk mengelola komunikasi pada jaringan kompleks seperti internet. Anda dapat menggunakan arsitektur berbasis REST untuk mendukung komunikasi berperforma tinggi dan andal sesuai skala. Anda dapat dengan mudah menerapkan dan memodifikasinya, membawa visibilitas dan *portabilitas* lintas platform ke semua sistem API.

Developer API dapat merancang API menggunakan beberapa arsitektur yang berbeda. API yang mengikuti gaya arsitektur REST disebut sebagai API REST. Layanan web yang menerapkan arsitektur REST disebut sebagai layanan web *Restful*. Istilah API *Restful* umumnya merujuk pada API web *Restful*. Namun, Anda dapat menggunakan istilah API REST dan API *Restful* secara bergantian.

2.10 API

Antarmuka Program Aplikasi (API) menentukan aturan yang harus Anda ikuti untuk berkomunikasi dengan sistem perangkat lunak lain. *Developer* mengekspos dan membuat API sehingga aplikasi lain dapat berkomunikasi dengan aplikasinya secara terprogram. Contohnya, aplikasi absensi mengekspos API yang meminta nama lengkap pegawai dan rentang tanggal. Saat menerima informasi ini, API memproses absensi pegawai secara *internal* dan mengembalikan jumlah jam kerja dalam rentang tanggal tersebut.

2.11 Restful API



Gambar 2.9 Restful API

(Sumber : <https://1.bp.blogspot.com/-rMhCq3PNrA/XDxO4XNcgII/AAAAAAAAACpQ/E6F3PanEx6IvIPecuXh2ABsSL-iTLbhCACLcBGAs/s1600/1.png>)

API Restful adalah antarmuka yang digunakan oleh dua sistem komputer untuk bertukar informasi secara aman melalui internet. Sebagian besar aplikasi bisnis harus berkomunikasi dengan aplikasi *internal* dan pihak ketiga lainnya untuk melakukan berbagai tugas. Misalnya, untuk menghasilkan slip gaji bulanan, sistem akun internal Anda harus berbagi data dengan sistem perbankan pelanggan Anda untuk mengotomatiskan tagihan dan berkomunikasi dengan aplikasi absensi *internal*. *API Restful* mendukung pertukaran informasi ini karena mengikuti standar komunikasi perangkat lunak yang aman, andal, dan efisien.

API Restful memiliki manfaat berikut:

a. Skalabilitas

Sistem yang menerapkan API REST dapat menskalakan secara efisien karena REST mengoptimalkan interaksi *klien-server*. *Statelessness* menghapus beban *server* karena *server* tidak perlu mempertahankan informasi permintaan *klien* di masa lalu. Pembuatan *cache* yang dikelola dengan baik secara parsial atau keseluruhan menghilangkan beberapa interaksi *klien-server*. Semua fitur ini mendukung skalabilitas tanpa menyebabkan kemacetan komunikasi yang mengurangi performa.

b. Fleksibilitas

Layanan web *Restful* mendukung pemisahan total *klien-server*. Layanan web *Restful* menyederhanakan dan memisahkan berbagai komponen *server* sehingga masing-masing bagian dapat berkembang secara mandiri. Perubahan *platform* atau teknologi pada aplikasi *server* tidak memengaruhi aplikasi *klien*. Kemampuan untuk melapisi fungsi aplikasi semakin meningkatkan fleksibilitas lebih jauh. Contohnya, *developer* dapat membuat perubahan pada lapisan basis data tanpa menulis ulang logika aplikasi.

c. Independensi

API REST *independen* terhadap teknologi yang digunakan. Anda dapat menulis baik aplikasi klien dan server dalam berbagai bahasa pemrograman tanpa memengaruhi desain API. Anda juga dapat mengubah teknologi mendasar di kedua sisi tanpa memengaruhi komunikasi.

Cara kerja API *Restful* sebagai berikut:

- 1) *Klien* mengirimkan permintaan ke *server*. *Klien* mengikuti dokumentasi API untuk memformat permintaan dalam format yang dipahami oleh *server*.
- 2) *Server* mengautentikasi *klien* dan mengonfirmasi bahwa *klien* memiliki hak untuk membuat permintaan.
- 3) *Server* menerima permintaan dan memproses secara *internal*.
- 4) *Server* mengembalikan respons kepada *klien*. *Respons* berisi informasi yang memberitahu klien jika permintaannya berhasil. *Respons* juga termasuk informasi apa saja yang diminta *klien*.

2.12 Database



Gambar 2.10 Database

(Sumber : <https://www.utopiccomputers.com/wp-content/uploads/2020/03/Gambar-Database.png>)

Pengertian *database* adalah sekumpulan data yang dikelola berdasarkan ketentuan tertentu yang saling berkaitan sehingga memudahkan dalam pengelolaannya. Dihimpun dari berbagai sumber, secara sederhana, *database* atau basis data merupakan sekumpulan data atau informasi yang tersimpan secara sistematis. *Database* memiliki peran penting dalam perangkat untuk mengumpulkan informasi, data, atau file secara terintegrasi.

Database membuat penyimpanan dan pengelolaan data menjadi lebih efisien. Adapun contoh *database* dapat dilihat dari pengembangan situs web. *Database* berwujud tabel yang terdiri dari kolom dan baris yang memuat atribut dan nilai tertentu. Adapun jumlah kolom dan baris dalam suatu *database* tergantung pada jumlah kategori atau jenis informasi yang perlu disimpan.

Fungsi *database* adalah untuk menghindari data ganda yang tersimpan. Suatu *database management system* (DBMS) dapat diatur supaya bisa mengenali duplikasi data ketika *diinput*. Namun selain untuk menghindari data ganda, *database* memiliki fungsi lainnya, antara lain:

- a. Mengelompokkan data dan informasi.
- b. Memudahkan dalam identifikasi data.
- c. Memudahkan proses akses, menyimpan, pembaharuan, dan penghapusan data.
- d. Menjadi alternatif terkait masalah penyimpanan ruang dalam suatu aplikasi.
- e. Menjaga kualitas data yang diakses sesuai *input*.
- f. Menunjang kinerja aplikasi yang memerlukan penyimpanan data.

Selain fungsi di atas, *database* bermanfaat untuk meminimalisasi *redundansi* data atau munculnya banyak data dalam file yang berbeda. *Database* dapat menunjang keamanan data. Hal tersebut lantaran sistem yang telah disusun secara aman melalui instrumen *password* sehingga data hanya bisa diakses oleh pihak yang diizinkan. Melalui sistem yang mampu menyeleksi data menjadi suatu kelompok berurutan, *database* dapat menghasilkan pencarian suatu informasi dengan lebih cepat. Namun, kecepatan tersebut juga dipengaruhi oleh jenis *database* yang digunakan.

2.13 DBMS



Gambar 2.11 DBMS

(Sumber : <https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcQPDk6L34NH7xQ0-VtIfWM-yuuoEerjCHXeYg&usqp=CAU>)

Database management system atau DBMS adalah suatu perangkat lunak yang dirancang untuk menghubungkan basis data dengan para *user*, agar pengelolaan data dapat berproses dengan baik.

Sesuai pengertiannya, DBMS adalah sistem yang bertugas sebagai *database engine*, *schema database*, dan pengelolaan data agar proses pengorganisasian dan manajemennya juga bisa berjalan lancar.

DBMS juga bisa diartikan sebagai suatu perantara *visual* yang dibutuhkan untuk melakukan kegiatan membaca, mengorganisir, memperbarui, serta menghapus data yang sudah tersimpan dalam *database* agar lebih mudah.

Ketika ingin mengoperasikan DBMS, maka *user* harus menggunakan bahasa komputer yang memang sesuai dengan sistem terkait. Terdapat dua jenis bahasa komputer yang disarankan, agar bisa berkomunikasi melalui DBMS, yaitu:

- a. *Data Manipulation Language*, bahasa ini umumnya digunakan untuk sebuah perintah yang akan memanipulasi data dalam basis data. Perintah ini berbentuk seperti pengambilan data, mengubah data, menghapus data dan juga menambah data.
- b. *Data Definition Language*, bahasa ini digunakan untuk mengubah serta membuat struktur dari suatu objek dalam basis data seperti *index*, *table*, *views*, *schema* dan lain sebagainya.

Sebagai suatu management *database*, DBMS memiliki beberapa fungsi, yaitu:

- a. **Mempercepat Akses Data**
Fungsi yang pertama adalah untuk mempercepat akses data dan memungkinkan *user* untuk memperoleh hasil *query* mereka secara akurat dan cepat.
Sistem manajemen basis data ini tentunya dapat meningkatkan produktivitas kerja yang terkait dengan *database*.
- b. **Mengurangi terjadinya *Inkonsistensi* Data**
Fungsi selanjutnya DBMS adalah *inkonsistensi* yang terjadi pada suatu file yang berada di beberapa tempat sekaligus. Perannya adalah untuk memastikan data yang sudah tersimpan, sudah berupa versi baru.
- c. **Untuk Keamanan Data**
Database harus terjaga keamanannya, agar dapat berjalan dengan baik. Pengaturan otorisasi *user* harus difasilitasi dengan fitur otorisasi akses dengan penggunaan *password* dan *username*.
- d. **Memanipulasi Data**
Suatu basis data biasanya memiliki cakupan informasi yang besar, dan tersimpan dalam bentuk dokumen juga tabel. DBMS adalah suatu informasi yang terletak beserta dengan kaitan dari informasi tersebut.

e. Untuk Meningkatkan Integritas Data

Dalam DBMS, suatu pengelolaan *database* ini harus dipastikan sudah akurat dan tersusun secara konsisten dengan baik. Adanya sistem pengelolaan basis data, memastikan agar data lebih tersusun dan konsisten.

f. Backup Data

DBMS adalah suatu sistem yang dapat berfungsi sebagai *backup* data. Hal tersebut sangat berguna untuk mencegah kehilangan data yang penting. Sistem secara otomatis sudah menyiapkan cadangan data tersebut.

g. Mendefinisikan Data dan Kaitannya

Database ini sudah memiliki informasi dengan cakupan yang besar dan tersimpan dalam bentuk dokumen atau suatu tabel. DBMS dapat membantu dalam mendefinisikan suatu informasi terkait dokumen tersebut.

Kelebihan DBMS adalah Mampu Meningkatkan Kualitas Data *Sharing*, dimana *user* bisa dengan mudah melakukan akses data secara rapi. Dapat meningkatkan efisiensi kerja sama dengan tim yang melakukan kolaborasi secara real time. Lebih mudah dalam pengambilan keputusan, dengan data yang tertata sangat rapi, dapat meningkatkan kualitas informasi data tersebut, sehingga mempengaruhi keputusan yang diambil.

Kekurangan DBMS adalah perlu biaya operasional, dalam konfigurasi *hardware* dan *software* yang cukup rumit, perlu ditangani oleh pekerja yang ahli dan memiliki keterampilan tertentu. Biaya operasional akan dibutuhkan dalam hal ini. Jika suatu organisasi memilih untuk menggunakan DBMS, maka diperlukan sebuah sistem yang mampu mengelola semua aspek *database*, teknis dan *non-teknis*. Frekuensi *Update*, fitur-fitur baru akan selalu diperbarui untuk DBMS ini, fitur terbaru juga ada yang dijual terpisah dan terkadang harus seimbang dengan *update hardware* juga.

2.14 XAMPP



Gambar 2.12 XAMPP

(Sumber : <https://i1.wp.com/prajasa.com/wp-content/uploads/2016/09/xampp-logo.png?ssl=1>)

XAMPP adalah *software* atau aplikasi komputer yang banyak digunakan dalam dunia web developer yang juga bisa dipelajari untuk membuat *Website*. XAMPP adalah perangkat lunak berbasis web *server* yang bersifat *open source* (bebas) serta mendukung di berbagai sistem operasi seperti OS Linux, OS Windows, Mac OS, dan juga Solaris.

XAMPP bisa dilakukan untuk menghemat anggaran karena mampu menggantikan peran web *hosting* dengan cara menyimpan file *Website* ke dalam *hosting* lokal agar bisa dipanggil lewat *browser*. *Software* XAMPP dikembangkan oleh tim bernama Apache *Friends* pada tahun 2002, yang bisa didapatkan secara gratis dengan label GNU (*General Public License*).

XAMPP adalah singkatan dari X (*cross platform*), A (Apache), M (MySQL/MariaDB), P (PHP), dan P (Perl) yang adalah program-program yang tersedia di *software* ini.

Fungsi utama XAMPP adalah sebagai server lokal untuk menyimpan berbagai jenis data *Website* yang sedang dalam proses pengembangan.

a. Mengakses dan Memodifikasi *Database* PhpMyAdmin

XAMPP bisa digunakan untuk mengatur halaman *database* pada PhpMyAdmin, dengan hanya mengakses server lokal komputer tanpa koneksi internet. PhpMyAdmin memungkinkan kamu untuk bebas mengedit, menghapus, memperbarui atau menambahkan *user*, dan juga *database* dengan mudah.

b. Menguji Fitur dan Mengakses *Website* tanpa Internet

XAMPP bisa digunakan untuk menguji kinerja fitur juga menampilkan konten yang ada di dalam *Website* kepada orang lain tanpa koneksi internet, cukup dengan

mengakses XAMPP *control panel*. XAMPP dapat bekerja secara *offline* seperti web *hosting* biasa namun tidak bisa diakses banyak orang, sehingga dapat digunakan untuk mempelajari WordPress tanpa koneksi internet sehingga lebih mudah dalam proses pengerjaan *front end* dan *back end*. XAMPP biasanya digunakan oleh mahasiswa ataupun pelajar untuk melihat hasil desain *website* sebelum dipublikasikan.

c. Menjalankan Laravel

XAMPP bisa menjalankan Laravel (*Framework* milik PHP yang berfungsi untuk mempermudah *programmer* mengembangkan tampilan *Website*) melalui perangkat komputer. Sehingga dapat melakukan modifikasi kode program atau *script* dan juga membuat fitur baru dengan lebih cepat dan lebih mudah.

2.15 MySQL



Gambar 2.13 MySQL

(Sumber : <https://idwebhost.com/blog/wp-content/uploads/2013/08/SQL.png>)

MySQL adalah sebuah DBMS (*Database Management System*) menggunakan perintah SQL (*Structured Query Language*) yang banyak digunakan saat ini dalam pembuatan aplikasi berbasis *Website*. MySQL dibagi menjadi dua lisensi, pertama adalah *Free Software* dimana perangkat lunak dapat diakses oleh siapa saja. Dan kedua adalah *Shareware* dimana perangkat lunak berpemilik memiliki batasan dalam penggunaannya.

MySQL termasuk ke dalam RDBMS (*Relational Database Management System*). Sehingga, menggunakan tabel, kolom, baris, di dalam struktur *database* - nya. Jadi, dalam proses pengambilan data menggunakan metode *relational database*. Dan juga menjadi penghubung antara perangkat lunak dan *database server*.

Secara garis besar, fungsi dari MySQL adalah untuk membuat dan mengelola *database* pada sisi server yang memuat berbagai informasi dengan menggunakan

bahasa SQL. Fungsi lain yang dimiliki adalah memudahkan pengguna dalam mengakses data berisi informasi dalam bentuk *String* (teks), yang dapat diakses secara personal maupun publik dalam web.

Hampir seluruh penyedia server web atau host menyediakan fasilitas untuk MySQL dalam pengembangan aplikasi berbasis *Website* untuk dikelola oleh *web developer*. Kemudian, antarmuka dari MySQL adalah PHPMyAdmin. Yang berfungsi untuk menghubungkan antara bahasa pemrograman PHP dengan MySQL untuk proses pengelolaan basis data pada web.

Beberapa kelebihan dari MySQL, diukur dari segi penggunaan dan fitur yang dimiliki, adalah sebagai berikut:

a. Bersifat *Open Source*

Untuk penggunaan dari MySQL dapat diakses oleh siapa saja dan kapan saja. Karena memiliki dua lisensi, yaitu *Free Software* dan *Shareware*. Selain itu juga termasuk dalam sistem manajemen *database* yang disediakan secara gratis. Serta, memiliki banyak komunitas dan dokumentasi untuk membantu anda dalam mengembangkan aplikasi menggunakan *database server*.

b. Mendukung Penggunaan *Multi User*

MySQL juga dapat digunakan secara bersama – sama dalam satu waktu karena bersifat *Multi User*. Sehingga, anda tidak perlu khawatir ketika mengakses basis data secara bersamaan. Keuntungannya disini adalah, tim developer dapat mengerjakan *database* secara bersamaan (*team work*).

c. Terintegrasi dengan Bahasa Pemrograman yang lain

Kelebihan lain adalah dapat terintegrasi dengan bahasa pemrograman yang lain seperti pada PHP. Dengan menggunakan PHPMyAdmin sebagai perantara, anda sudah bisa membuat basis data pada web server dengan lebih efektif. Kemudian, dapat digunakan juga untuk kebutuhan pengembangan perangkat lunak dengan menggunakan *Framework* seperti Laravel, dan CodeIgniter.

d. Sajian Tipe Data yang Bervariatif

Tipe data yang disajikan dalam MySQL juga sangat variatif. Contohnya adalah tipe data *varchar*, *integer*, *float*, *double*, *date*, *timestamp*, dll. Dalam hal ini, anda dapat mengelompokkan berbagai data sesuai dengan kriteria informasi secara lebih akurat dan efektif. Pengelolaan *database* dalam server juga akan terbantu dengan adanya banyak jenis tipe data untuk mengembangkan perangkat lunak yang kompleks dan terstruktur.

e. Struktur Tabel Cenderung Fleksibel

Jika dibandingkan dengan struktur tabel dari *database* yang lain seperti Oracle dan Postgre, MySQL tergolong lebih mudah. Mengapa demikian? Karena sebagai contoh dalam proses eksekusi pada *ALTER TABLE* lebih mudah digunakan dan cepat diproses.

f. Sistem Keamanan yang Telah Terjamin

MySQL dapat mengatur hak akses pengguna (*user*) dengan *enkripsi* tinggi. Selain itu, terdapat beberapa fitur keamanan yang lain seperti, level nama pada *host*, dan pembuatan *subnet mask*. Jadi, bukan berarti dengan perangkat lunak yang bersifat *open source* membuat keamanan menjadi buruk.

g. Tidak Membutuhkan Kapasitas RAM yang Besar

MySQL dapat diinstal pada perangkat dengan kapasitas RAM terbatas. Jika dibandingkan dengan *database* lain, dapat dijalankan dengan kapasitas memori yang kecil. Sangat cocok digunakan oleh setiap orang untuk membantu pengembangan aplikasi dengan konsumsi RAM yang rendah.

h. Perangkat Lunak *Portable*

Kelebihan berikutnya yang dimiliki oleh MySQL adalah perangkat lunak bersifat *portable*. *Portable* disini maksudnya adalah sistem basis data dapat diolah dalam berbagai *platform* (*multi platform*) Sehingga, dapat dijalankan baik dari sisi Windows, Linux, maupun Mac OS.

i. *Support* untuk Spesifikasi *Hardwre* yang Rendah

MySQL sangat mendukung untuk perangkat *hardware* yang memiliki spesifikasi rendah. Jadi, jangan khawatir untuk anda yang memiliki spek *PC* / laptop yang masih rendah. Dengan menggunakan DBMS ini, anda juga bisa membuat sistem basis data untuk *server* dalam sebuah *Website*.

j. Memiliki Performa yang Baik

Kelebihan terakhir yang dimiliki adalah dari sisi performa yang dimilikinya. Kecepatan dalam penanganan *queri* sangat cepat. Kemudian, juga dapat menangani proses SQL per satuan waktu.

Selain memiliki banyak keuntungan, pastinya juga memiliki beberapa kekurangan. Berikut merupakan beberapa kekurangan dari sisi fitur dan penggunaan.

a. Kurang Cocok untuk Mengembangkan Aplikasi berbasis Game dan Mobile

Untuk anda yang sedang mengembangkan aplikasi untuk membuat game dan mobile, kami sarankan untuk tidak menggunakan MySQL. Karena, manajemen *database* yang dimiliki kurang cocok untuk pengelolaan aplikasi tersebut. Lebih mudah untuk menggunakan *database* server dengan metode *Cloud* yang lebih *real time*.

b. *Technical Support* yang Kurang Mendukung

Technical support yang dimiliki oleh MySQL sendiri dirasa kurang mendukung. Karena, belum menyediakan *support* yang baik dari segi teknisnya. Sehingga, *user* mengalami kesulitan saat menghubungi *technical support*.

c. Kurang Mendukung untuk Pemakaian *Database* dengan Kapasitas Besar

Kurang cocok dan tidak direkomendasikan untuk mengelola basis data dalam kapasitas besar. Apabila anda, menginginkan sebuah *database server* yang dapat menampung data dalam jumlah besar, maka anda dapat mencoba dengan menggunakan *Big Data* atau *BigQuery*. Karena, MySQL dikhususkan supaya ramah dengan perangkat yang memiliki spesifikasi rendah.

2.16 Apache



Gambar 2.14 Apache

(Sumber : <https://gudangssl.id/blog/wp-content/uploads/2021/08/apache-1.png>)

Apache adalah *software web server yang open source*. Web server yang satu ini berfungsi untuk meningkatkan *user experience* saat pengguna mengakses Website Anda.

Kabar baiknya, selain Apache adalah *software web server open source*, web server ini juga gratis loh, *GudPoeple*.

Software web server yang dikembangkan dan dikelola oleh lembaga *non-profit* bernama *Apache Software Foundation* ini memungkinkan para pemilik Website untuk mengelola Website agar semuanya dapat berjalan dengan mulus.

Caranya adalah dengan menjadi perantara antara server dengan sisi *klien*. Web server nantinya akan memproses file ke dalam bermacam bahasa pemrograman seperti Java, PHP, Python dan lainnya.

Jadi, pada dasarnya *software* ini akan menerima permintaan untuk mengakses sebuah halaman Website dan melakukan pemeriksaan keamanan pada permintaan HTTP lalu membawa Anda ke halaman website yang dituju. Namun, bisa saja halaman yang ingin dituju akan meminta server untuk menjalankan beberapa modul ekstra saat membuat dokumen tergantung pada halaman yang Anda akses.

Apache di sini berfungsi sebagai salah satu cara untuk mengkoneksikan jaringan antara *klien* dengan server menggunakan protokol TCP/IP. Sebetulnya, Apache juga dapat digunakan dengan protokol lain, tapi umumnya digunakan dengan protokol HTTP/S (*Hyper Text Transfer Protocol Secure*). HTTP/S ini digunakan untuk mendefinisikan bagaimana format suatu pesan dan bagaimana pesan tersebut ditransmisikan pada web.

Proses tersebut dapat dilakukan dengan instruksi untuk *browser* dan server mengenai cara merespon berbagai *request* dan *command*. Dengan protokol HTTP/S, transmisinya akan melalui *port* 443, sedangkan untuk *unsecured protocol*

akan melalui protokol 80. Setelah pesan tersebut sampai ke penerimanya, maka nantinya akan ada pemberitahuan atau ACK message yang pada dasarnya memberitahu pengirim pesan bahwa data mereka sudah sampai.

Jika ada *error* saat menerima data atau ada *package* yang hilang saat *transit*, sisi *klien* akan mengirimkan pesan NAK atau *Not Acknowledge message*. Pesan tersebut memberikan notifikasi bahwa data harus ditransmisikan ulang.

Di awal sudah disebutkan bahwa Apache adalah *software web server* yang gratis dan *open source*. Tapi, adakah kelebihan-kelebihan lain dari Apache? Di bagian ini kami akan menjelaskan beberapa di antaranya.

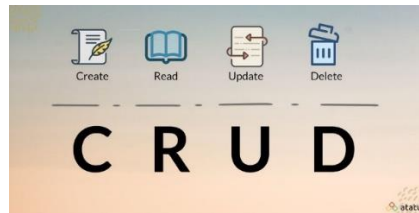
- a. *Source code* Apache dapat digunakan secara gratis dan dapat diakses oleh siapapun tanpa lisensi.
- b. Dokumentasi resmi Apache sangat berguna dan selalu berkembang
- c. Apache sangat fleksibel namun tetap stabil.
- d. Dapat berfungsi baik di *server* Unix maupun Windows karena lintas platform.
- e. Diperbarui secara regular.
- f. Penginstalan mudah.
- g. Dapat digunakan di berbagai CMS seperti WordPress, Joomla, Drupal, dll.
- h. Beberapa *Website* dapat dijalankan dari server yang sama dengan menggunakan Apache web server.
- i. Perubahan yang dibuat akan langsung direkam tanpa harus merestart server.
- j. Mampu ditambah dengan berbagai fitur dan modul.
- k. Dapat dimodifikasi sehingga kode-kodenya bisa disesuaikan dan bisa membetulkan *error*.
- l. Dapat digunakan bersamaan dengan web *Framework* seperti Laravel, Django, Vaadin, dll.
- m. Didukung oleh berbagai Bahasa pemrograman seperti Python, PHP, Perl, dll.
- n. Terdapat SSL dan TLS untuk meningkatkan keamanannya.

Dari sekian banyaknya kelebihan dari Apache, ternyata *software web server* ini juga memiliki kekurangan. Namun jangan khawatir, kekurangannya tidak lebih banyak dari kelebihan Apache.

- a. Men-*disable* atau menonaktifkan *module* dan *service* yang dirasa tidak perlu membuat Apache menjadi rentan terhadap serangan keamanan.

- b. Membuat *personalized protocol* akan memunculkan bug baru.
- c. Jika tidak dilakukan dengan tepat, memodifikasi fitur dan konfigurasi Apache juga dapat mengancam keamanannya.
- d. *Website* dengan Apache masih belum bisa menerima trafik yang sangat tinggi.

2.17 CRUD



Gambar 2.15 CRUD

(Sumber : <https://www.atatus.com/glossary/content/images/size/w960/2021/07/CRUD.jpeg>)

CRUD adalah singkatan yang berasal dari *Create*, *Read*, *Update*, dan *Delete*, dimana keempat istilah tersebut merupakan fungsi utama yang nantinya diimplementasikan ke dalam basis data. Empat poin tersebut mengindikasikan bahwa fungsi utama melekat pada penggunaan *database relasional* beserta aplikasi yang mengelolanya, seperti Oracle, MySQL, SQL Server, dan lain – lain.

Jika dihubungkan dengan tampilan antarmuka (*interface*), maka peran CRUD sebagai fasilitator berkaitan dengan tampilan pencarian dan perubahan informasi dalam bentuk formulir, tabel, atau laporan. Nantinya, akan ditampilkan dalam browser atau aplikasi pada perangkat komputer *user*. Istilah ini pertama kali diperkenalkan oleh James Martin pada tahun 1983 dalam bukunya yang berjudul “Managing the *Database Environment*”.

Terdapat empat poin penting dari akronim fungsi CRUD untuk mengembangkan perangkat lunak, baik berbasis web maupun mobile.

a. *Create*

Fungsi CRUD yang pertama adalah *create*, dimana anda dapat memungkinkan untuk membuat *record* baru pada sistem basis data. Jika anda sering menggunakan SQL, maka sering disebut dengan istilah *insert*.

Sederhananya, anda dapat membuat tabel atau data baru sesuai atribut dengan memanggil fungsi *create*. Akan tetapi, biasanya hanya posisi administrator saja yang dapat menambahkan atribut lain ke dalam tabel itu sendiri.

b. *Read*

Fungsi yang kedua adalah *read*, berarti memungkinkan anda untuk mencari atau mengambil data tertentu yang berada di dalam tabel dengan membaca nilainya. Fungsi *read* mempunyai kesamaan dengan fungsi *search* yang biasa anda temukan dalam berbagai perangkat lunak.

Hal yang perlu anda lakukan adalah dengan menggunakan kata kunci (*keyword*) untuk dapat menemukan file *record* dengan bantuan filter data berdasarkan kriteria tertentu.

c. *Update*

Fungsi CRUD yang ketiga adalah *update*, dimana berfungsi untuk memodifikasi data atau *record* yang telah tersimpan di dalam *database*. Namun, anda perlu untuk mengubah beberapa informasi terlebih dahulu agar dapat mengubah *record* sesuai kebutuhan anda. Untuk pengisian *update* data anda juga perlu menyesuaikan nilai atribut sesuai dengan form yang tersedia agar tidak ada kesalahan saat pemrosesan data di dalam *server*.

d. *Delete*

Fungsi yang terakhir adalah *delete*, dimana ketika anda tidak membutuhkan sebuah *record* lagi, maka data tersebut perlu untuk dihapus. Sehingga, anda perlu untuk menggunakan fungsi *delete* untuk memproses aktivitas tersebut.

Beberapa *software* terkait *database relasional* mengizinkan anda untuk menggunakan *soft* dan *hard delete*. Untuk *soft delete* berfungsi untuk memperbarui status baris yang menunjukkan bahwa data akan dihapus meskipun informasi tersebut tetap ada. Sedangkan, untuk *hard delete* bertujuan untuk menghapus catatan pada basis data secara permanen.

Terdapat beberapa keuntungan apabila anda menggunakan CRUD ini, diantaranya adalah sebagai berikut.

a. Digunakan oleh Berbagai Bahasa Pemrograman

Fungsi CRUD terkenal dengan fleksibilitasnya, sehingga dapat digunakan dan diimplementasikan ke dalam banyak bahasa pemrograman, seperti JavaScript, PHP,

SQL, Ajax, Java, Python, dan lain – lain. Selain itu, juga dapat dipergunakan dalam berbagai *Framework*, seperti *Codeigniter* dan *Laravel*.

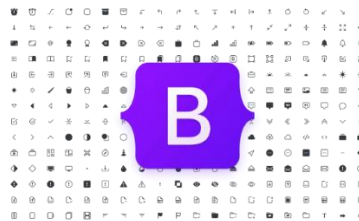
b. Operasi Pemrograman yang Cukup Kompleks

Kelebihan yang kedua, CRUD juga termasuk fungsi esensial untuk dapat dipahami. Sehingga, ada beberapa saat ketika anda akan merasakan kesulitan saat memahami pemrograman komputer sebelum anda benar – benar paham terkait materi ini.

c. Mendukung Tampilan UI Berjalan dengan Baik

Keuntungan yang ketiga, fungsi CRUD juga mendukung tampilan *user interface* menjadi lebih terkontrol, akurat, dan tepat sasaran. Dimana, berbagai tipe data dapat ter- *define* sesuai dengan atribut dan nilai masing – masing.

2.18 Bootstrap



Gambar 2.16 Bootstrap

(Sumber : <https://qnp.co.id/wp-content/uploads/2022/08/bootstrap-icons.png>)

Bootstrap adalah sebuah *Framework* HTML, CSS, dan JavaScript yang berfokus untuk menyederhanakan pengembangan halaman web atau *Website*. Pada umumnya, Bootstrap digunakan untuk mengimplementasikan berbagai pilihan warna, ukuran, font, dan layout yang ada dalam *Framework* tersebut ke dalam sebuah *Website*.

Pada awal pengembangannya, Bootstrap memiliki nama lain, yaitu Twitter Blueprint. Mark Otto dan Jacob Thornton mengembangkan Bootstrap untuk meningkatkan konsistensi dalam berbagai alat internal yang digunakan dalam pengembangan *Website*.

Sebagai sebuah *Framework*, Bootstrap menyediakan *template* untuk mendefinisikan *style* dasar seluruh elemen HTML dalam sebuah *Website*. Hal ini

akan mempermudah dalam pembuatan *Website* dengan keseragaman *visual* tanpa harus mendefinisikan *style attribute* untuk setiap elemen HTML secara berulang-ulang.

Selain elemen HTML dan CSS, Bootstrap juga menawarkan beberapa komponen JavaScript dalam bentuk plugin jQuery. Adanya plugin jQuery ini memungkinkan pengguna Bootstrap untuk menggunakan beberapa fitur *interaktif* seperti dialog *box*, *tooltips*, *carousel*, dan sebagainya.

Berikut ini adalah beberapa fungsi utama Bootstrap:

a. Membuat *Website mobile-friendly*

Sistem *grid* yang dimiliki oleh Bootstrap memudahkan *developer* untuk mengembangkan *website responsive* dan *mobile friendly* yang akan tampil dengan baik pada berbagai ukuran layar. Bootstrap memiliki sistem *grid* yang akan mempermudah penyusunan elemen-elemen *Website* berdasarkan ukuran layar yang menampilkannya.

b. Menghemat waktu pembuatan *Website*

Bootstrap adalah sebuah *Framework* yang dapat digunakan untuk menyingkat waktu pengembangan *Website* dengan berbagai kemudahan yang ditawarkannya. Ketika kamu menggunakan Bootstrap, kamu dapat mengatur tampilan setiap elemen HTML secara seragam dari satu pengaturan saja. Jadi, tak perlu repot-repot mengubah atribut ratusan elemen HTML.

c. Meminimalisir *bug* antar *browser*

Salah satu masalah terbesar dalam pengembangan *Website* adalah performanya yang tidak konsisten pada *browser* yang berbeda. Bootstrap dapat membantu meminimalisir munculnya *bug* yang terjadi karena faktor kompatibilitas *browser* yang digunakan oleh pengunjung.

d. Fitur kustomisasi yang berlimpah

Jika kamu tidak puas dengan *template* desain yang ditawarkan oleh Bootstrap, kamu tetap dapat memodifikasi file CSS sesuai dengan

keinginanmu. Hal ini memungkinkan kamu untuk menciptakan *Website* dengan tampilan yang unik, namun tidak ingin membuat kode CSS *custom* dari nol.

e. Meningkatkan konsistensi desain

Dalam proses *development* sebuah *Website*, inkonsistensi antara *front-end* dan *back-end* merupakan masalah yang sangat sering terjadi. Para pendiri Bootstrap bertujuan untuk menemukan solusi dari masalah tersebut.

Penggunaan *Framework* Bootstrap memungkinkan tim *front-end* dan *back-end* untuk mengacu pada referensi yang sama, sehingga akan menciptakan keselarasan yang menyeluruh.

f. Bersifat *Open-source*

Bootstrap adalah sebuah *Framework* yang bersifat *open-source*. Artinya, seluruh *developer*, termasuk kamu, dapat memodifikasinya sesuai kebutuhan tanpa perlu membayar lisensi apapun.

Selain itu, para *developer* yang menggunakan Bootstrap juga selalu menambahkan fitur-fitur baru yang akan membuat *Framework* ini terus berkembang.

g. Ketersediaan *resource* & dukungan

Selain menawarkan *resource* yang melimpah melalui situs resmi mereka, Bootstrap juga memiliki dukungan komunitas yang sangat luas. Jadi, kamu dapat dengan mudah mencari solusi dari masalah saat mengembangkan *Website* dengan Bootstrap.

Meskipun Bootstrap memiliki banyak sekali kelebihan dalam mempercepat proses pengembangan *Website*, bukan berarti Bootstrap tak memiliki kekurangan. Berikut ini adalah beberapa kekurangan Bootstrap *framework* untuk pengembangan *Website*:

a. Kemiripan dengan *Website* Bootstrap lain

Bootstrap adalah sebuah *framework* untuk menerapkan gaya *visual* pada sebuah *Website*. *Framework* ini dikembangkan tujuan mempermudah pengembangan *Website* dengan menggunakan sejenis '*template*' dalam kode yang digunakan.

Maka dari itu, sebagian besar *Website* yang menggunakan Bootstrap *framework* akan memiliki tampilan *visual* yang serupa. Jika ingin memiliki *Website* Bootstrap dengan tampilan unik, developer diharuskan untuk melakukan modifikasi yang cukup banyak terhadap kode bawaannya.

b. Perlu proses pembelajaran

Meskipun Bootstrap dapat dipelajari dengan mudah, kamu tetap harus menghabiskan waktu untuk mempelajari cara penggunaannya dan menghafal berbagai CSS class yang dapat digunakan.

Selain itu, kamu juga akan perlu waktu untuk beradaptasi dan bereksperimen dengan sistem *grid* yang digunakan oleh Bootstrap. Namun dokumentasi Bootstrap yang sangat lengkap dan rapih akan sangat membantu pengguna baru untuk beradaptasi ke lingkungan Bootstrap.

c. Dapat memperlambat *Website*

Bootstrap adalah sebuah *framework* yang mempermudah pembuatan *Website responsif* dan *mobile-friendly*. Namun jika tidak melalui proses optimasi yang baik, *Website* Bootstrap cenderung memiliki performa yang lebih buruk dalam segi load time untuk para penggunanya.

Selain itu, file yang dibuat oleh Bootstrap dapat mencapai ukuran yang relatif lebih besar dibandingkan dengan *Framework* lain, yang tentunya dapat memperlambat *Website*.

2.19 Entity Relationship Diagram (ERD)

ERD atau *entity relationship diagram* adalah istilah dari bahasa Inggris yang artinya diagram hubungan entitas. ERD pun sering disebut dengan model ER atau ER Diagram.

Sederhananya, *entity relationship diagram* adalah salah satu jenis diagram yang sifatnya lebih struktural dan bisa digunakan untuk dimanfaatkan dalam suatu desain pada suatu *database* ataupun pada sebuah *business plan*.

Selain itu, terdapat banyak sekali komponen yang menjadikan *entity relationship diagram*, seperti *konektor* dan juga simbol yang berbeda-beda. Nantinya, komponen ini akan melakukan *visualisasi* pada dua informasi yang dianggap penting. Informasi pertama adalah entitas utama yang terdapat di dalam ruang lingkup suatu sistem. Sedangkan untuk informasi kedua adalah hubungan yang terdapat di berbagai *entitas* itu sendiri.

Berdasarkan penjelasan ini, mungkin sebagian dari Anda sudah bisa memahami sistem *entity relationship diagram* ini, karena di dalamnya hanya mencerminkan hubungan antar setiap *entitas* saja.

Saat kita membahas tentang entitas di dalam *entity relationship diagram*, maka umumnya entitas yang dimaksud adalah suatu objek yang berhubungan dengan bisnis. Nah, objek ini bisa berwujud ataupun berbentuk benda seperti produk barang ataupun bentuk tidak berwujud lainnya seperti *entry log*, dan lain-lain. Lalu, *entity relationship diagram* adalah tentang bagaimana cara entitas tersebut bisa saling berhubungan antara yang satu dengan yang lainnya di dalam suatu sistem.

Terdapat banyak sekali komponen yang terdapat di dalam susunan *entity relationship diagram*, beberapa diantaranya adalah sebagai berikut:

- a. Entitas

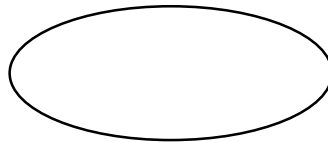


Gambar 2.17 Simbol Entitas

Entitas adalah sekumpulan objek yang nantinya akan diidentifikasi. Ketika sedang membuat ERD, umumnya suatu entitas akan digambarkan dalam suatu simbol persegi panjang. Disisi lain, entitas

yang lemah akan digambarkan dengan simbol persegi panjang yang kecil di dalam persegi panjang yang lebar. Masing-masing entitas sudah tentu mempunyai perbedaan. Bila ternyata ada kesamaan, maka entitas tersebut tidak perlu dicantumkan.

b. Atribut



Gambar 2.18 Simbol Atribut

Setiap entitas selalu mempunyai elemen ataupun atribut agar bisa menjelaskannya dengan baik. Umumnya, atribut akan digambarkan sebagai simbol. Di dalam *entity relationship diagram*, terdapat beberapa jenis atribut, seperti atribut simple, atribut kunci, atribut multival, atribut gabungan, dan juga atribut derivatif.

c. Relasi



Gambar 2.19 Simbol Relasi

Relasi ataupun hubungan adalah suatu tingkat ketertarikan pada beberapa entitas dari gabungan lainnya. umumnya, *relasi* ini akan dicerminkan dalam simbol berbentuk belah ketupat. Di dalam *entity relationship diagram*, relasi ini akan dibagi menjadi beberapa jenis, seperti *One to One*, *One to Many*, dan juga *Many to Many*.

Entity relationship diagram (ERD) didesain dengan berbagai fungsi dan tujuannya masing-masing. Namun, fungsi umum dari *entity relationship diagram* adalah sebagai berikut:

- a. Membantu menganalisis suatu *database* dengan cara yang lebih cepat dan juga lebih murah.

- b. Mampu menjalankan relasi antar setiap data yang mempunyai keterkaitan dengan berdasarkan objek yang dihubungkan dengan suatu relasi khusus.
- c. Membantu menjalankan dokumentasi data yang terdapat dalam suatu *database* dengan cara melakukan analisis dan identifikasi pada setiap objek ataupun entitas serta relasinya.
- d. Melakukan suatu pengujian model yang sebelumnya sudah dibuat.

Cara Membuat *Entity Relationship Diagram* (ERD) adalah sebagai berikut :

- a. Melakukan Identifikasi Entitas

Bila Anda ingin membuat model *entity relationship diagram*, maka hal pertama yang harus Anda kerjakan adalah mengidentifikasi seluruh entitas yang nantinya akan Anda gunakan.

Anda bisa memasukkan seluruh entitas di dalam kolom persegi dengan cara mencantumkan deskripsi yang singkat terkait informasi yang nantinya ingin Anda simpan. Dengan cara ini, maka seluruh entitas yang ingin Anda gunakan bisa memiliki sistem dan juga struktur yang baik. Hal ini mampu menghindari adanya entitas yang tertinggal.

Selain itu, cobalah untuk menyediakan bagian kosong agar bisa menambah informasi tambahan terkait entitas tersebut.

- b. Melakukan Identifikasi dan Menjelaskan *Relasi* atau Hubungan Entitas

Langkah selanjutnya adalah dengan melakukan identifikasi ulang dan menjelaskan hubungan antar setiap entitas. Namun sebelumnya, Anda harus terlebih dulu mencari dua entitas yang memang mempunyai hubungan.

Bila sudah, sambungkanlah dengan suatu garis dan berikanlah penjelasan tentang bagaimana hubungan di dalamnya bisa terjadi. Selanjutnya, berikanlah simbol khusus *relasi* berbentuk belah ketupat di dalam deskripsi hubungan tersebut.

c. Tambahkan Atribut

Langkah selanjutnya adalah menambahkan atribut kunci pada setiap entitas. Gunakanlah simbol berbentuk oval dan tambahkanlah beberapa atribut sesuai dengan fungsi atribut yang sudah ditentukan.

2.20 Data Flow Diagram (DFD)

DFD adalah suatu diagram yang menggambarkan aliran data dari sebuah proses yang sering disebut dengan sistem informasi. Di dalam *data flow diagram* juga menyediakan informasi mengenai *input* dan *output* dari tiap entitas dan proses itu sendiri.

Dalam diagram alir data juga tidak mempunyai kontrol terhadap *flow*-nya, sehingga tidak adanya aturan terkait keputusan atau pengulangan. Bentuk penggambaran berupa data *flowchart* dengan skema yang lebih spesifik. Menurut Kenneth Kozar, tujuan dari adanya DFD sendiri adalah sebagai penyedia atau menjembatani antara pengguna dengan sistem.

Data flow diagram berbeda dengan UML (*Unified Modelling Language*), dimana hal mendasar yang menjadi pembeda antara kedua skema tersebut terletak pada *flow* dan *objective* penyampaian informasi di dalamnya.

Secara fundamental, terdapat tiga fungsi dari pembuatan diagram alir data untuk kebutuhan *software development*. Berikut ini merupakan penjelasan dari masing-masing fungsi di bawah ini.

a. Menyampaikan Rancangan Sistem

Dengan pembuatan DFD, maka proses penyampaian informasi menjadi lebih mudah dengan tampilan *visual* yang simple dan dapat dimengerti oleh tiap *stakeholder*. Dimana, data yang disajikan mampu menggambarkan alur data secara terstruktur dengan pendekatan yang lebih efisien.

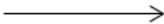
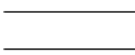
b. Menggambarkan Suatu Sistem

Fungsi yang kedua, DFD dapat membantu proses penggambaran sistem sebagai jaringan *funksional*. Maksudnya adalah, di dalam jaringan terdapat berbagai komponen yang saling terhubung menggunakan alur data.

c. Perancangan Model

Fungsi yang terakhir, diagram ini juga dapat membuat rancangan model baru dengan menekankan pada fungsi sistem tertentu. Hal tersebut dapat dimanfaatkan untuk melihat bagian yang lebih detail dari diagram alir data tersebut.

Terdapat beberapa simbol utama untuk menyusun sebuah rangkaian DFD yang tepat, diantaranya adalah sebagai berikut.

Gambar	Nama	Keterangan
	Data Flow/Aliran Data	Menjelaskan arah data/informasi entitas, proses, data store
	Process/Proses	Menjelaskan proses dalam sebuah DFD
	Data storage/Simpanan data	Menjelaskan tempat penyimpanan suatu data/informasi
	External entity, boundary	Menjelaskan suatu entitas luar pada sebuah DFD

Gambar 2.16 Simbol DFD

(Sumber : [http://1.bp.blogspot.com/-MLBald-](http://1.bp.blogspot.com/-MLBald-TiB8/U0hxyzCKvYPI/AAAAAAAAAJU/BWZ4ZFpzLkk/s1600/DFD+Versi+Yourdan+DeMarco.PNG)

[TiB8/U0hxyzCKvYPI/AAAAAAAAAJU/BWZ4ZFpzLkk/s1600/DFD+Versi+Yourdan+DeMarco.PNG](http://1.bp.blogspot.com/-MLBald-TiB8/U0hxyzCKvYPI/AAAAAAAAAJU/BWZ4ZFpzLkk/s1600/DFD+Versi+Yourdan+DeMarco.PNG))

a. *Data Store*

Data store adalah file untuk menyimpan data yang digunakan untuk proses selanjutnya. Dapat dikatakan juga, sama seperti basis data (*database*). Pada umumnya, data store berupa tabel yang dapat diolah, serta mampu terhubung dengan setidaknya satu masukan dan satu keluaran. Penggambaran atau simbol *data store* berupa dua garis sejajar.

b. *Data Flow*

Data flow merupakan arus data yang mengalir antara *terminator*, *proses*, dan *data store*. Data flow digambarkan dengan simbol tanda panah, dan

fungsi utamanya adalah untuk mengalirkan informasi dari satu sistem ke sistem yang lain.

c. *External Entity*

External entity atau lebih sering disebut dengan *terminator* merupakan pihak di luar sistem, dapat berupa individu, divisi, perusahaan, atau sistem yang lainnya. *Terminator* dapat memberikan masukan atau keluaran terhadap sistem. Simbol dari *external entity* dilambangkan dengan persegi panjang atau kotak.

d. *Process*

Process dilakukan oleh mesin dengan mengubah *input* menjadi *output* dengan format yang berbeda. Simbol proses digambarkan dalam bentuk lingkaran, oval, atau persegi panjang dengan tambahan sudut bundar.

Data flow diagram terbagi menjadi tiga jenis, dimana setiap bagian memiliki peran dan fungsinya masing-masing. Untuk pembuatannya sendiri dapat menyesuaikan kebutuhan proyek dari manajemen tim-nya.

a. Diagram Level 0 (Diagram *Konteks*)

Diagram *konteks* atau level 0 merupakan diagram dengan tingkatan paling rendah, dimana menggambarkan sistem berinteraksi dengan entitas eksternal. Pada diagram *konteks* akan diberi nomor untuk setiap proses yang berjalan, dimulai dari angka 0 terlebih dahulu.

Jadi, untuk setiap aliran data akan langsung diarahkan menuju sistem. Ciri dari diagram level 0 terletak pada tidak adanya informasi yang terkait data yang tersimpan pada *data store*.

b. Diagram Level 1

DFD level 1 merupakan lanjutan dari diagram *konteks* karena setiap proses yang berjalan akan diperinci pada tingkatan ini sehingga proses utama akan dipecah menjadi sub-sub proses yang lebih kecil lagi.

c. Diagram Level 2

DFD level 2 merupakan tingkat lanjutan dari level yang sebelumnya, dimana pada fase ini akan dijelaskan lebih detail terkait tiap prosesnya. Namun, untuk tingkatan ini jarang sekali dikerjakan dan lebih banyak hanya menerapkan dua level di bawahnya saja.

Setelah mengetahui jenis dan notasi diagram alir data, selanjutnya masuk pada pembahasan mengenai bagaimana cara membuat data flow diagram yang baik dan benar.

a. *Data Store* Harus Diproses

Pertama, yang perlu anda perhatikan adalah setiap data yang tersimpan di dalam *data store* harus diproses lebih lanjut untuk dijadikan sebagai keluaran (*output*).

b. Menentukan Jumlah *Input* dan *Output*

Kedua, pada setiap DFD setidaknya mempunyai satu *inputan* dan satu keluaran karena diagram alir data harus mencerminkan alur sistem dari tahap awal hingga akhir.

c. Hubungan pada *Data Store*

Ketiga, setiap data store harus saling terhubung dengan setidaknya satu *input* dan satu *output* agar dapat menyimpan data yang masuk menuju sistem.

d. Letak Posisi Proses

Aturan terakhir, setiap proses yang telah terjadi pada diagram alir data harus melalui proses untuk menghasilkan *output* yang sesuai.

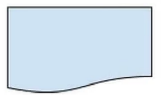
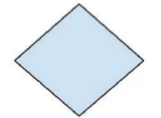
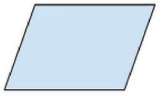
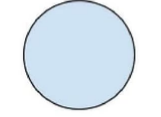
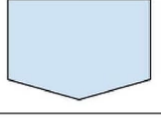
2.21 Flowchart

Flowchart atau bagan alur adalah diagram yang menampilkan langkah-langkah dan keputusan untuk melakukan sebuah proses dari suatu program. Setiap langkah digambarkan dalam bentuk diagram dan dihubungkan dengan garis atau arah panah.

Flowchart berperan penting dalam memutuskan sebuah langkah atau fungsionalitas dari sebuah proyek pembuatan program yang melibatkan banyak orang sekaligus. Selain itu dengan menggunakan bagan alur proses dari sebuah program akan lebih jelas, ringkas, dan mengurangi kemungkinan untuk salah penafsiran. Penggunaan *flowchart* dalam dunia pemrograman juga merupakan cara yang bagus untuk menghubungkan antara kebutuhan teknis dan *non-teknis*.

Fungsi utama dari *flowchart* adalah memberi gambaran jalannya sebuah program dari satu proses ke proses lainnya. Sehingga, alur program menjadi mudah dipahami oleh semua orang. Selain itu, fungsi lain dari *flowchart* adalah untuk menyederhanakan rangkaian prosedur agar memudahkan pemahaman terhadap informasi tersebut.

Simbol- simbol *flowchart* sebagai berikut :

No.	Simbol Flowchart	Nama	Arti Simbol Flowchart
1		<i>Terminator</i>	Awal atau akhir konsep (prosedur)
2		<i>Process</i>	Proses operasional
3		<i>Document</i>	Dokumen atau laporan berupa <i>print out</i>
4		<i>Decision</i>	Keputusan atau sub-point. Garis yang terhubung dengan bentuk <i>decision</i> merujuk pada situasi-situasi yang berbeda sesuai dengan keputusan yang digambarkan
5		Data	Input dan Output (Contohnya, Input: feedback dari pelanggan. Output: desain produk baru)
6		<i>On-Page Reference/Connector</i>	Penghubung alur dalam halaman yang sama
7		<i>Off-Page Reference/Off-Page Connector</i>	Penghubung alur dalam halaman yang berbeda
8		<i>Flow</i>	Arah alur dalam konsep (prosedur)

Gambar 2.21 Simbol *Flowchart*

(Sumber : <https://info.populix.co/wp-content/uploads/2022/04/2-51.webp>)

Flowchart sendiri terdiri dari lima jenis, masing-masing jenis memiliki karakteristik dalam penggunaannya. Berikut adalah jenis-jenisnya:

a. *Flowchart* dokumen

Pertama ada *flowchart* dokumen (*document flowchart*) atau bisa juga disebut dengan *paperwork flowchart*. *Flowchart* dokumen berfungsi untuk menelusuri alur form dari satu bagian ke bagian yang lain, termasuk bagaimana laporan diproses, dicatat, dan disimpan.

b. *Flowchart* program

Selanjutnya kita akan membahas *flowchart* program. *Flowchart* ini menggambarkan secara rinci prosedur dari proses program. *Flowchart* program terdiri dari dua macam, antara lain: *flowchart* logika program (*program logic flowchart*) dan *flowchart* program komputer terinci (*detailed computer program flowchart*).

c. *Flowchart* proses

Flowchart proses adalah cara penggambaran rekayasa industrial dengan cara merinci dan menganalisis langkah-langkah selanjutnya dalam suatu prosedur atau sistem.

d. *Flowchart* sistem

Yang keempat ada *flowchart* sistem. *Flowchart* sistem adalah *flowchart* yang menampilkan tahapan atau proses kerja yang sedang berlangsung di dalam sistem secara menyeluruh. Selain itu *flowchart* sistem juga menguraikan urutan dari setiap prosedur yang ada di dalam sistem.

e. *Flowchart* skematik

Terakhir ada *flowchart* skematik. *Flowchart* ini menampilkan alur prosedur suatu sistem, hampir sama dengan *flowchart* sistem. Namun, ada perbedaan dalam penggunaan simbol-simbol dalam menggambarkan alur. Selain simbol-simbol, *flowchart* skematik juga

menggunakan gambar-gambar komputer serta peralatan lainnya untuk mempermudah dalam pembacaan *flowchart* untuk orang awam.

2.22 Visual Studio Code



Gambar 2.22 Visual Studio Code

(Sumber : https://cdn.icon-icons.com/icons2/2107/PNG/512/file_type_vscode_icon_130084.png)

Visual Studio Code adalah sebuah *code editor* gratis yang bisa dijalankan di perangkat desktop berbasis Windows, Linux, dan MacOS. *Code editor* ini dikembangkan oleh salah satu raksasa teknologi dunia, Microsoft.

Visual Code adalah *software editor* yang powerful, tapi tetap ringan ketika digunakan. Ia bisa dipakai untuk membuat dan mengedit *source code* berbagai bahasa pemrograman. Misalnya, seperti JavaScript, TypeScript, dan Node.js.

Bahkan, *Visual Code Studio* juga kompatibel dengan bahasa dan *runtime environment* lain, seperti PHP, Python, Java, dan .NET. Hal ini berkat ekosistemnya yang luas dan ketersediaan *extension* yang melimpah.

Maka tidak heran, jika *Visual Code* adalah *code editor* terpopuler saat ini. Hal ini terbukti dari *survey Stack Overflow*, di mana *software* yang satu ini menduduki peringkat teratas dengan persentase pengguna 71.06%.

Berikut ini beberapa fitur *Visual Code Studio* yang menjadikannya sebagai *software editor* paling banyak digunakan saat ini:

a. Basic Editing

Sesuai fungsinya sebagai *code editor*, kemampuan *Visual Code Studio* untuk *coding* tidak perlu diragukan lagi. Ia punya semua yang Anda butuhkan. Mulai dari *Keyboard Shortcuts*, *Multiple Selection*, hingga *Column Selection*.

Bahkan, *Visual Code* juga menambahkan fitur *Auto Save* dan *Hot Exit*. Keduanya berfungsi untuk menyimpan file secara *otomatis*. Hal ini

berguna untuk mencegah hal-hal yang tidak diinginkan, seperti lupa menyimpan file.

b. IntelliSense

IntelliSense adalah fitur *Visual Studio* yang dapat memudahkan proses *coding*. Cara kerjanya mirip dengan *Autocomplete*, yakni menyarankan keseluruhan kata berdasarkan apa yang Anda ketik.

Nah, *IntelliSense* tersedia secara default untuk JavaScript, TypeScript, JSON, HTML, dan CSS. Namun jangan khawatir, Anda dapat menambahkan *IntelliSense* untuk bahasa pemrograman lain lewat *Extension Marketplace*.

c. Debugging

Fitur kunci lainnya di *Visual Code* adalah *Debugging*. Fitur ini berguna untuk memudahkan Anda dalam mengedit, meng-*compile*, dan mengeksekusi kode berulang kali (*looping*).

Secara *default*, *Visual Studio* menyediakan fitur *Debugging* untuk Node.js. Namun, *Debugging* untuk bahasa pemrograman lain tetap dapat Anda unduh melalui *Extension Marketplace*.

d. Extension Marketplace

Extension Marketplace adalah fitur yang membuat *Visual Code Studio* unggul jauh dari para pesaingnya. Dengan *Extension*, Anda bisa dengan mudah *install tools, debuggers*, bahkan bahasa pemrograman tambahan.

Dengan begitu, proses *website development* Anda jadi lebih cepat. Menariknya lagi, *Visual Code* memungkinkan para pembuat *Extension* berkontribusi secara fungsional, guna menghindari masalah kompatibilitas.

e. *Github Integration*

Fitur unggulan lain dari *Visual Code* adalah *integrasi Github*, platform *manajemen project* terpopuler di dunia. Di sini, Anda dapat berbagi *code* dan berkolaborasi dengan rekan kerja, tanpa perlu berpindah *software*. Cara menggunakannya juga sangat mudah. Anda hanya perlu menyiapkan akun *Github*, kemudian menginstall *Github Pull Request and Issues* melalui *Extension Marketplace*.

Setelah mengenal fitur-fitur *Visual Code Studio*, mari mengenal kelebihan memakai *Visual Studio Code* untuk *coding*:

a. Tersedia di Banyak *Platform*

Visual Code dapat diinstall di *platform* Linux, MacOS, dan Windows. Jadi, tidak ada lagi kendala dukungan terhadap perangkat yang Anda gunakan.

b. Fitur yang Lengkap

Seperti yang sudah dijelaskan, kelengkapan fitur menjadi keunggulan utama *Visual Studio*. Hal ini berkat adanya *Extension Marketplace*, sehingga Anda bisa menambah fitur-fiturnya dengan bebas.

c. Performa Secepat Kilat

Meski dijejali banyak *Extension*, performa *Visual Code Studio* tetap tangguh. Sebab, *Extension* yang tersedia telah dioptimasi sedemikian rupa, sehingga tidak mempengaruhi kinerja *code editor* ini.

d. Dukungan Arsitektur Terbaik

Visual Studio menggunakan berbagai teknologi terbaik di dalamnya. Seperti *Electron* untuk pengembangan JavaScript dan Node.js, Monaco Cloud Editor untuk HTML, Roslyn untuk .NET, dan sebagainya.

2.23 Postman



Gambar 2.23 Postman

(Sumber : <https://www.vectorlogo.zone/logos/getpostman/getpostman-ar21.png>)

Dengan kata sederhana, *Postman* adalah aplikasi komputer yang digunakan untuk pengujian API. *Postman* mengirim permintaan API ke *server web* dan menerima *respons*, apa pun itu. Tidak ada pekerjaan tambahan atau pengaturan kerangka kerja yang diperlukan saat mengirim dan menerima permintaan di *Postman*. Banyak digunakan oleh Penguji dan Pengembang untuk pengujian aplikasi yang lebih baik. Mudah diintegrasikan dengan *Continuous Integration (CI)* & *Continuous Development Pipeline* Anda.

Postman dengan sejumlah fitur dan kemudahan kerjanya, telah digunakan oleh jutaan penguji. Menggunakan antarmuka yang sederhana dan ramah pengguna, Anda dapat dengan mudah mengirim permintaan, cukup isi data yang diperlukan, pilih metode HTTP dan tekan tombol "Kirim". Fitur lain yang paling banyak digunakan adalah Otomatisasi, yang memungkinkan Anda mengatur pengujian dan menulis rangkaian pengujian.

Meskipun gratis untuk digunakan, *Postman* menawarkan Anda versi berbayar dengan fitur pro seperti akses banyak pengguna, Kontrol atas peran, akses, Otentikasi SSO, dll. Memungkinkan pengguna untuk mengatur lingkungan yang diperlukan, menulis spesifikasi dan akhirnya memantau setiap langkah, adalah semua itu membuat Alat Pengujian *Postman* ideal.

Postman adalah alat pengujian ramah pengguna yang memungkinkan Anda mengirim permintaan API dalam langkah yang lebih sedikit. Untuk memulainya, pada halaman arahan Anda, Anda akan melihat sejumlah tombol yang dirancang untuk *navigasi* yang mudah.

Mengklik tombol Baru, Anda dapat membuat permintaan atau koleksi baru. Saat di layar baru, Permintaan HTTP seperti *POST*, *GET*, *DELETE*, dll terdaftar dan dapat dipilih sesuai kebutuhan. Di sebelahnya adalah *URL* Permintaan, di mana Anda diharapkan untuk memasukkan *URL* titik akhir dan di akhir baris adalah

tombol Kirim, yang setelah diklik akan mengirim permintaan ke URL yang dimaksud dan menerima tanggapan.

Ini adalah dasar cara menggunakan *Postman*, tetapi *Postman* memiliki sejumlah fitur lain yang digunakan untuk pengujian yang lebih baik. Mengikuti tombol Kirim, tombol Simpan memungkinkan Anda untuk menyimpan permintaan saat ini agar tidak kehilangan perubahan apa pun. *Params*, di mana parameter dimasukkan dalam format Nilai Kunci,

Otorisasi diperlukan ketika Anda berurusan dengan *URL* yang tidak terbuka untuk umum dan untuk itu digunakan *token* pembawa atau Nama Pengguna & Kata Sandi. Tergantung pada kebutuhan, *Header* dengan tipe konten sebagai JSON dapat diatur. *Body* memungkinkan Anda untuk mengatur detail yang disesuaikan mungkin untuk permintaan *POST*. *Pre request Script* adalah *script* yang dijalankan sebelum *request*, dengan tujuan untuk memastikan bahwa *environment* diatur sesuai kebutuhan sehingga pengujian lebih lanjut dilakukan dengan benar. Tes terdiri dari skrip yang dijalankan saat permintaan sedang diproses, ini membantu Anda dalam memverifikasi status *respons* dan poin lainnya.

Impor memungkinkan Anda untuk mengimpor koleksi yang ada sementara *Runner* memungkinkan Anda untuk menjalankan tes otomatisasi.

Mengotomatiskan Pengujian API dapat dibagi dalam empat langkah dasar, dimulai dengan Pengujian Manual API, kemudian memahami *respons* API yang diterima, berdasarkan dua langkah sebelumnya Anda sekarang dapat menulis rangkaian pengujian dan akhirnya menjalankan rangkaian pengujian ini dari titik akhir yang berbeda.

Pada dasarnya, rangkaian pengujian ditulis atau dibuat sesuai dengan persyaratan parameter API dan detail lain yang diperlukan dan rangkaian pengujian ini dijalankan berulang kali, dari berbagai titik akhir. Skrip pengujian ini ditulis dalam JavaScript dan dapat dijalankan sebelum permintaan dibuat dan setelah *respons* diterima atas permintaan yang dibuat. Jumlah cuplikan tersedia bagi Anda untuk memulai pengujian.

Sekarang, untuk langkah terakhir, yang harus Anda lakukan adalah mengirim permintaan secara *manual*, tetapi kali ini setelah *respons* diterima untuk permintaan, *Postman* akan secara otomatis menjalankan tes tertulis. Tab Hasil Tes menunjukkan kepada Anda hasil tes.

Ini adalah contoh paling sederhana untuk mengotomatisasi pengujian di *Postman*, dengan rangkaian pengujian tambahan dan beberapa permintaan bersama dengan parameter dan otorisasi yang diperlukan, Anda dapat menjalankan pengujian yang kompleks.

BAB III

DESAIN DAN RANCANGAN

3.1 Analisis Kebutuhan

3.1.1 Perangkat dan Alat

Perangkat dan alat yang digunakan untuk merancang sistem aplikasi Parkir Kendaraan adalah sebagai berikut :

1. Laptop Acer TravelMate P2410-G2-M
2. Alat Tulis
3. Koneksi Internet

3.1.2 Software

Software yang digunakan untuk merancang dan menampilkan data Aplikasi Parkir Kendaraan adalah sebagai berikut:

1. Google Chrome
2. Text Editor *Visual Studio Code*
3. XAMPP
4. *Postman*

3.1.3 Library

Library yang digunakan pada sistem Aplikasi Parkir Kendaraan diantaranya sebagai berikut:

1. Bootstrap 4
2. *CodeIgniter 4*
3. Auth
4. *Oauth2*

3.1.4 Data

Data yang berkaitan dengan sistem Aplikasi Parkir Kendaraan diantaranya sebagai berikut:

1. Data Siswa SMK PGRI 03 Malang
2. Data Kendaraan
3. Data Satpam

4. Data tempat Parkir

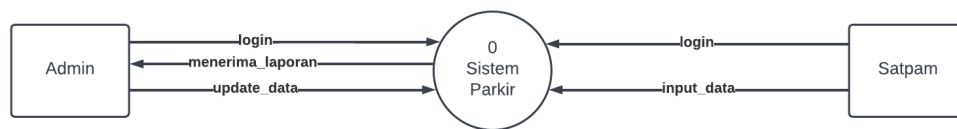
3.2 Analisis Perancangan Sistem

3.2.1 Deskripsi Aplikasi Parkir Kendaraan

Aplikasi parkir kendaraan merupakan sebuah aplikasi yang menyediakan data mengenai aktivitas kendaraan pada tempat parkir SMK PGRI 03 Malang. Aplikasi ini memuat informasi seputar pemilik kendaraan, nomor plat kendaraan, serta aktivitas kendaraan selama dalam lingkungan tempat parkir SMK PGRI 03 Malang. Aplikasi ini sangat berguna bagi para warga sekolah SMK PGRI 03 Malang agar dapat selalu memantau keadaan kendaraannya hanya melalui aplikasi yang disajikan dengan tampilan yang menarik dan mudah dipahami oleh pengguna.

3.2.2 DFD

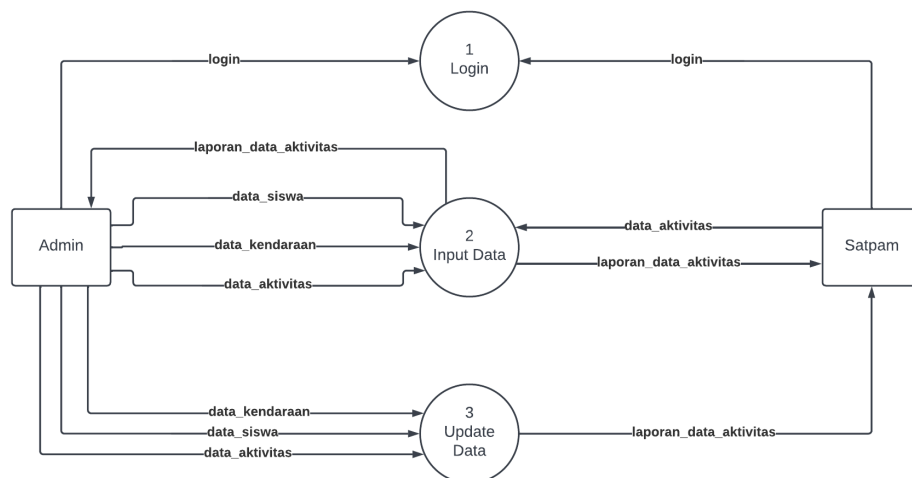
a. DFD Level 0 (Context)



Gambar 3.1 DVD Level 0

Pada DFD level 0 ini terdapat 2 entitas yaitu Admin dan Satpam. Selain itu juga digambarkan secara garis besar apa saja yang dapat dilakukan dan apa yang akan diterima oleh kedua entitas tersebut.

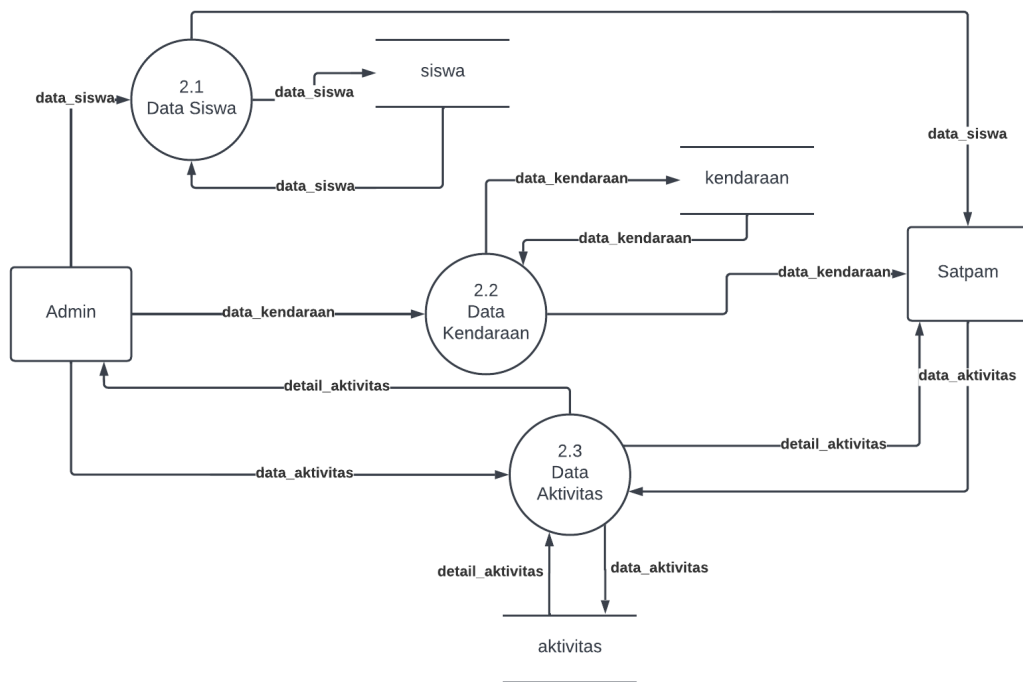
b. DFD Level 1



Gambar 3.2 DFD Level 1

Pada DFD level 1 ini berisi tentang data apa saja yang bias diakses oleh admin dan satpam. Juga diperlihatkan masing masing entitas mendapatkan apa dan melakukan apa pada sistem informasi tersebut.

c. DFD Level 2

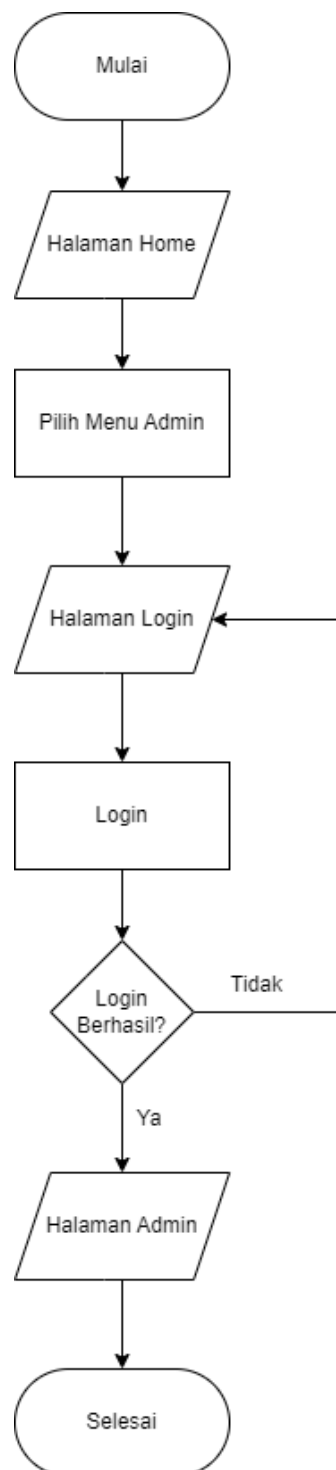


Gambar 3.3 DFD Level 2

Pada DFD level 1 berisi tentang CRUD master dan keuangan yang dimana datanya akan masuk ke masing masing *database*, juga diperlihatkan masing masing entitas mendapatkan apa dan melakukan apa pada sistem informasi tersebut.

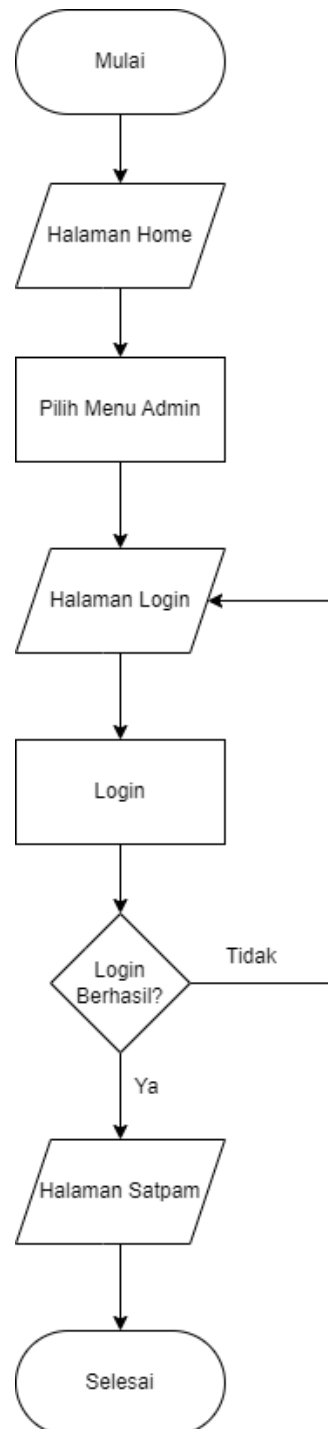
3.2.3 Flowchart

a. Flowchart Login Admin



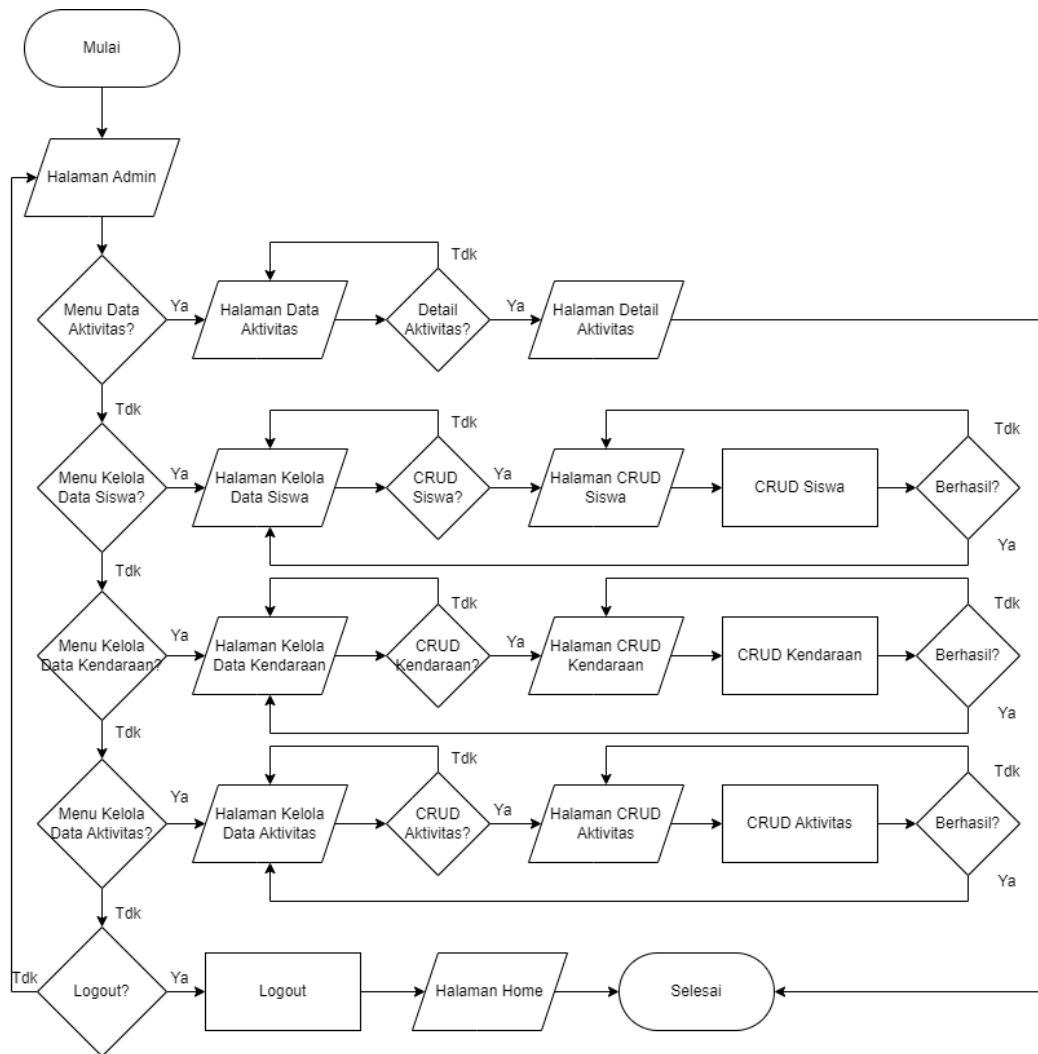
Gambar 3.4 Flowchart Login Admin

Pada *flowchart* ini menjelaskan bagaimana proses *login* admin.

b. Flowchart Login Satpam**Gambar 3.5** Flowchart Login Satpam

Pada *flowchart* ini menjelaskan bagaimana proses *login* Satpam.

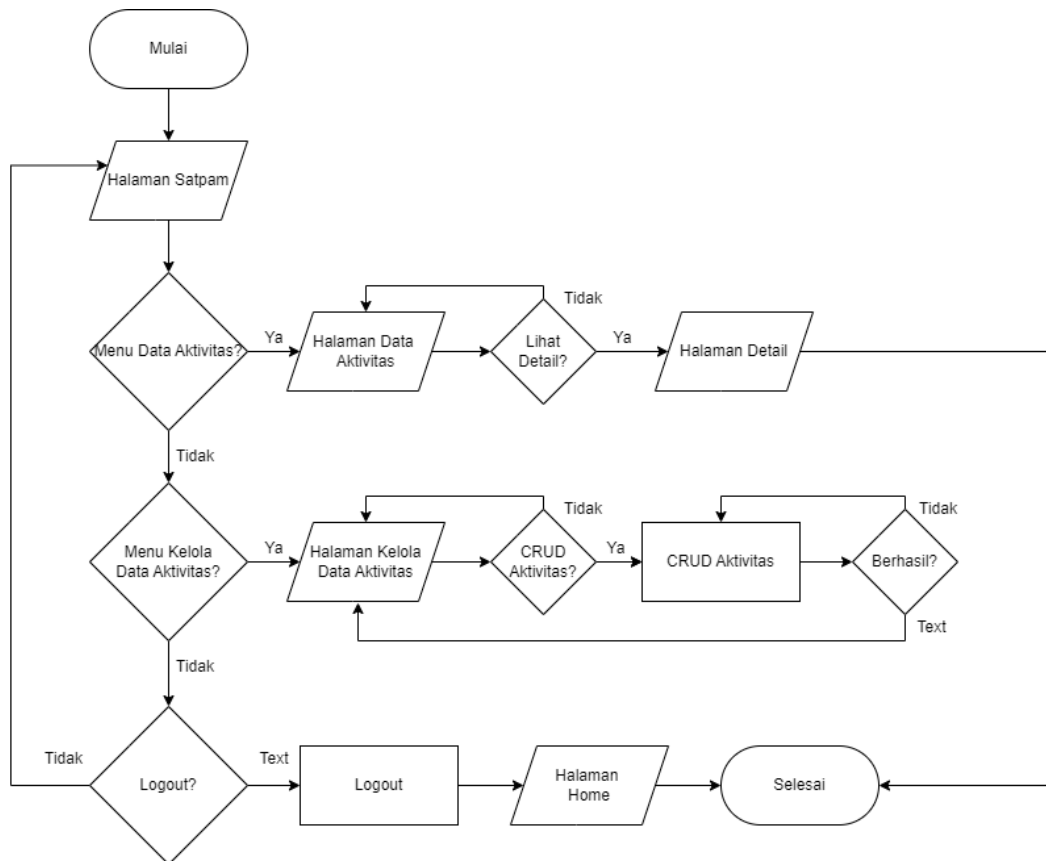
c. **Flowchart Halaman Admin**



Gambar 3.6 Flowchart Halaman Admin

Flowchart ini menjelaskan apa saja yang dapat dilakukan oleh admin pada halaman Admin.

d. Flowchart Halaman Satpam



Gambar 3.7 Flowchart Halaman Satpam

Flowchart ini menjelaskan apa saja yang dapat dilakukan oleh satpam pada halaman Satpam.

3.3 Rancangan Database

3.4.1 Database

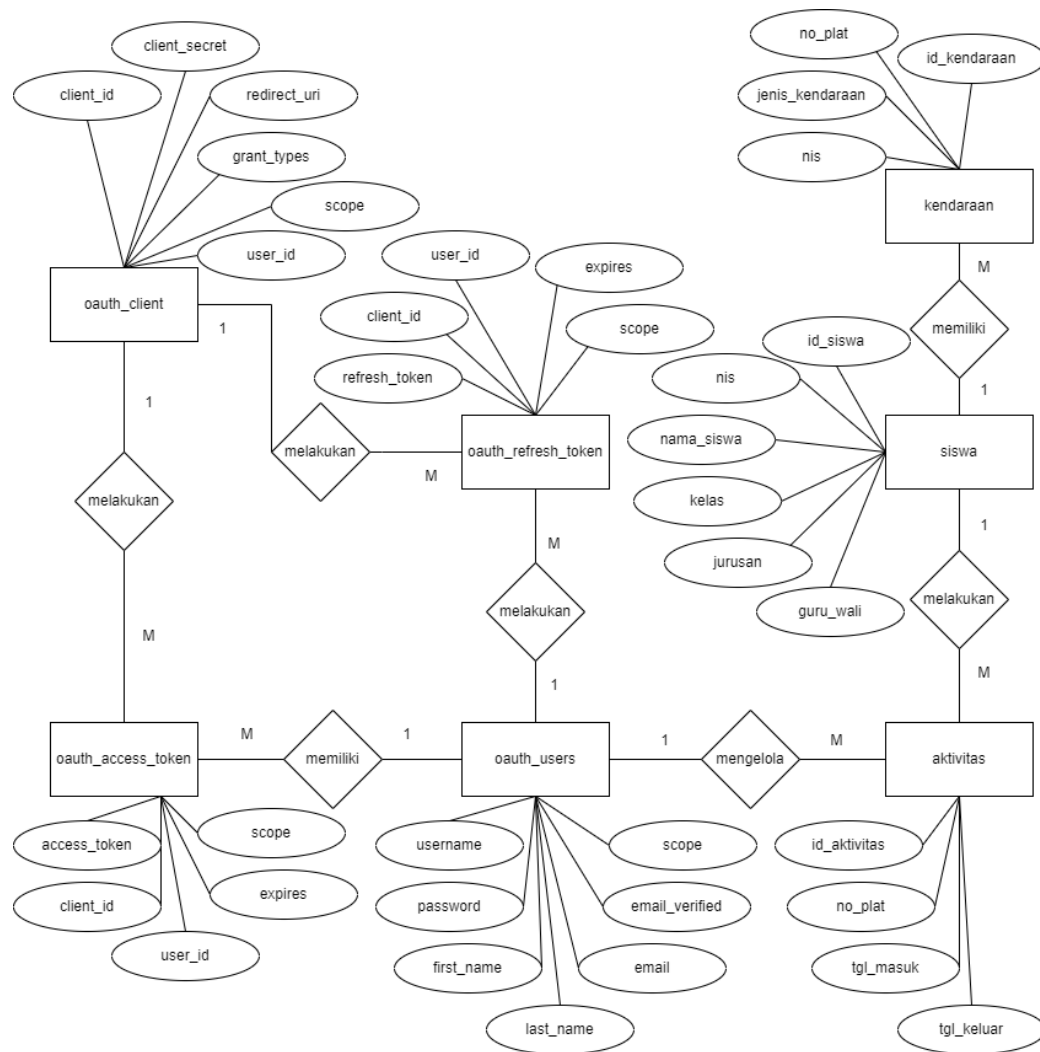
The screenshot shows the phpMyAdmin interface for a database named 'arestapi'. The left sidebar displays a tree view of databases, with 'arestapi' selected. The main panel shows the 'Structure' tab for the 'arestapi' database. A table with 7 rows is displayed, listing the tables in the database. Each row includes a checkbox, the table name, an 'Action' column with icons for Browse, Structure, Search, Insert, Empty, and Drop, and columns for Rows, Type, Collation, Size, and Overhead. The tables listed are: aktivitas (4 rows), kendaraan (10 rows), oauth_access_tokens (125 rows), oauth_clients (2 rows), oauth_refresh_tokens (125 rows), oauth_users (2 rows), and siswa (6 rows). A summary row at the bottom shows 7 tables with a total of 274 rows.

Table	Action	Rows	Type	Collation	Size	Overhead
☐ aktivitas		4	InnoDB	utf8mb4_general_ci	32.0 KIB	-
☐ kendaraan		10	InnoDB	utf8mb4_general_ci	48.0 KIB	-
☐ oauth_access_tokens		125	InnoDB	utf8mb4_general_ci	48.0 KIB	-
☐ oauth_clients		2	InnoDB	utf8mb4_general_ci	48.0 KIB	-
☐ oauth_refresh_tokens		125	InnoDB	utf8mb4_general_ci	48.0 KIB	-
☐ oauth_users		2	InnoDB	utf8mb4_general_ci	16.0 KIB	-
☐ siswa		6	InnoDB	utf8mb4_general_ci	32.0 KIB	-
7 tables	Sum	274	InnoDB	utf8mb4_general_ci	272.0 KIB	0 B

Gambar 3.8 Database

Database ini terdiri dari 7 tabel, yaitu *table aktivitas*, *table kendaraan*, *table oauth_access_token*, *table oauth clients*, *table oauth refresh token*, *table oauth user*, dan *table siswa*.

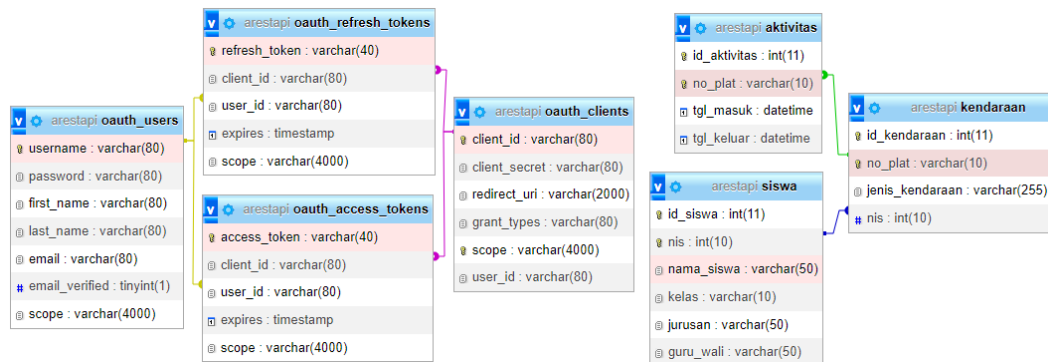
3.4.2 ERD



Gambar 3.9 ERD

Dalam ERD ini terdiri dari 7 tabel, yaitu *table* aktivitas, *table* kendaraan, *table* oauth_access_token, *table* oauth clients, *table* oauth refresh token, *table* oauth user, dan *table* siswa.

3.4.3 Relasi Tabel



Gambar 3.10 Relasi Tabel

Pada aplikasi parker kendaraan ini memiliki 7 tabel dan relasi antar tabel nya sesuai dengan gambar diatas.

3.4.4 Tabel

A. Tabel Siswa

Dalam *table* siswa terdapat 6 kolom yaitu, *id_siswa* sebagai primary key , *nis*, *nama_siswa*, *kelas*, *jurusan*, dan *guru_wali*.

Table 3.1 Siswa

Field	Tipe Data	Panjang	Key
Id Siswa	Int	11	PK AI
NIS	Int	10	Unique
Nama_siswa	Varchar	50	Not Null
Kelas	Varchar	5	Not Null
Jurusan	Varchar	25	Not Null
Guru Wali	Varchar	50	Not Null

B. Tabel Kendaraan

Pada *table* kendaraan terdapat 4 kolom yaitu, *id_kendaraan* sebagai primary key, *no_plat*, *jenis_kendaraan*, dan *nis*.

Table 3.2 Kendaraan

Field	Tipe Data	Panjang	Key
Id Kendaraan	Int	11	PK AI
No. Polisi Keendaraan	Varchar	10	Unique
Jenis Kendaraan	Varchar	30	Not Null
NIS	Int	10	FK

C. Tabel Aktivitas

Table aktivitas terdapat 4 kolom yaitu, *id_aktivitas* sebagai primary key, *no_plat*, *tgl_masuk*, dan *tgl_keluar*.

Tabel 3.3 Aktivitas

Field	Tipe Data	Panjang	Key
Id Aktivitas	Int	11	PK
No. Polisi Kendaraan	Varchar	10	FK
Tgl Masuk	Datetime	-	Current_timestamp()
Tgl Keluar	Datetime	-	Null

D. Tabel Oauth Access Tokens

Table *oauth access token* terdapat 5 kolom yaitu, *access_token* sebagai primary key, *client_id*, *user_id*, *expires*, dan *scope*.

Table 3.4 Oauth Access Token

Field	Tipe Data	Panjang	Key
Access Token	Varchar	40	PK
Client Id	Varchar	80	FK
User Id	Varchar	80	FK
Expires	Timestamp	-	Current_timestamp()
Scope	Varchar	4000	Null

E. Tabel Oauth Clients

Tabel *oauth clients* terdiri dari 6 kolom yaitu, *client_id* sebagai primary key, *client_secret*, *redirect_uri*, *grant_types*, *scope*, dan *user_id*.

Table 3.5 Oauth Clients

Field	Tipe Data	Panjang	Key
Client Id	Varchar	80	PK
Client Secret	Varchar	80	Null
Redirect Uri	Varchar	2000	Null
Grant Types	Varchar	80	Null
Scope	Varchar	4000	Null
User Id	Varchar	80	Null

F. Tabel *Oauth Refresh Tokens*

Table oauth refresh token memiliki 5 kolom di dalamnya yaitu, *refresh_token* sebagai primary key, *client_id*, *user_id*, *expires*, dan *scope*.

Table 3.6 *Oauth Refresh Token*

Field	Tipe Data	Panjang	Key
<i>Refresh Token</i>	Varchar	40	PK
<i>Client Id</i>	Varchar	80	Not Null
<i>User Id</i>	Varchar	80	Null
<i>Expires</i>	Timestamp	-	Current_timestamp()
<i>Scope</i>	Varchar	4000	Null

G. Tabel *Oauth Users*

Pada *table oauth users* terdapat 7 kolom yaitu, *username* sebagai primary key, *password*, *first_name*, *last_name*, *email*, *email_verified*, dan *scope*.

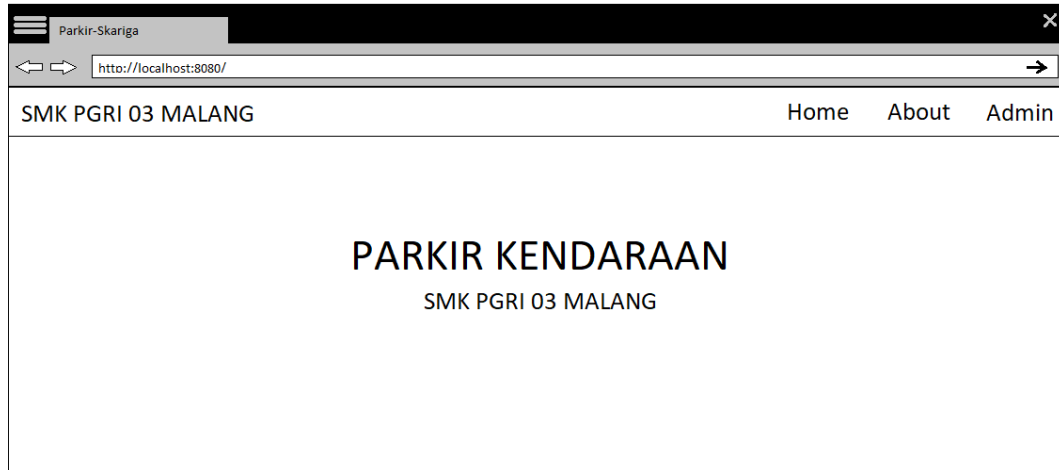
Tabel 3.7 *Oauth Users*

Field	Tipe Data	Panjang	Key
<i>Username</i>	Varchar	80	Not Null
<i>Password</i>	Varchar	80	Null
<i>First Name</i>	Varchar	80	Null
<i>Last Name</i>	Varchar	80	Null
<i>Email</i>	Varchar	80	Null
<i>Email Verified</i>	Tinyint	1	Null
<i>Scope</i>	Varchar	4000	Null

3.4 Rancangan Tampilan Aplikasi

A. Home

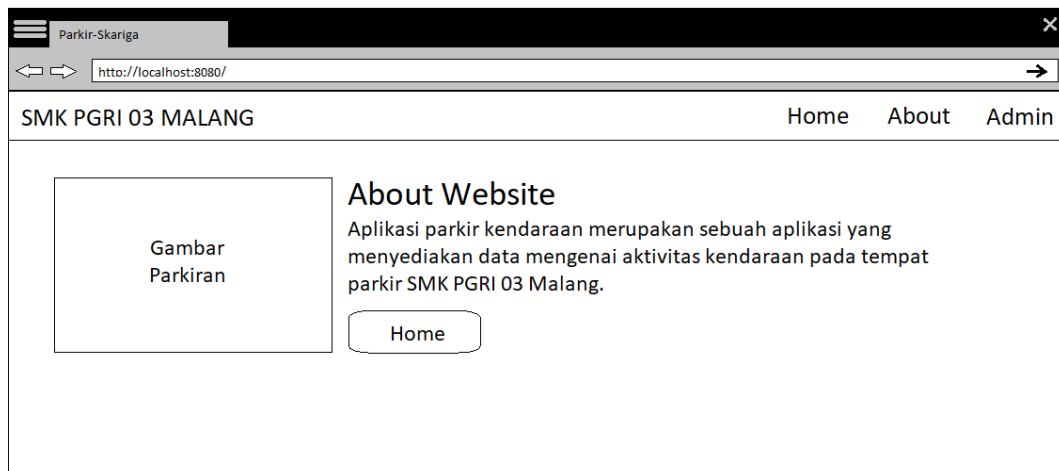
Desain tampilan *Home* yang berisi judul aplikasi, dan beberapa fitur untuk mengakses halaman lainnya.



Gambar 3.11 Login

B. About

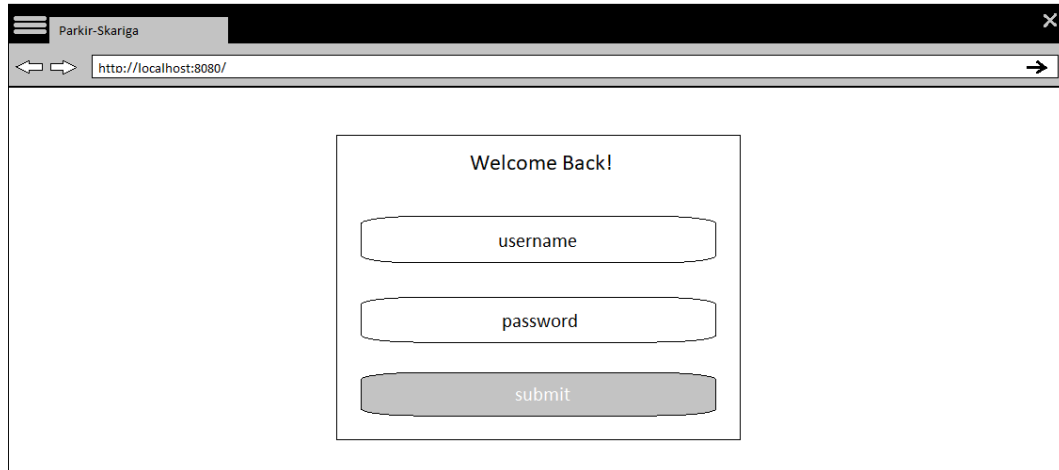
Desain halaman *About* berisi tentang penjelasan mengenai cara kerja aplikasi yang dibuat serta terdapat button untuk kembali ke halaman *home*.



Gambar 3.12 About

C. Login

Desain form *login* berisi inputan untuk admin dan satpam yang berisi *username* dan *password*.



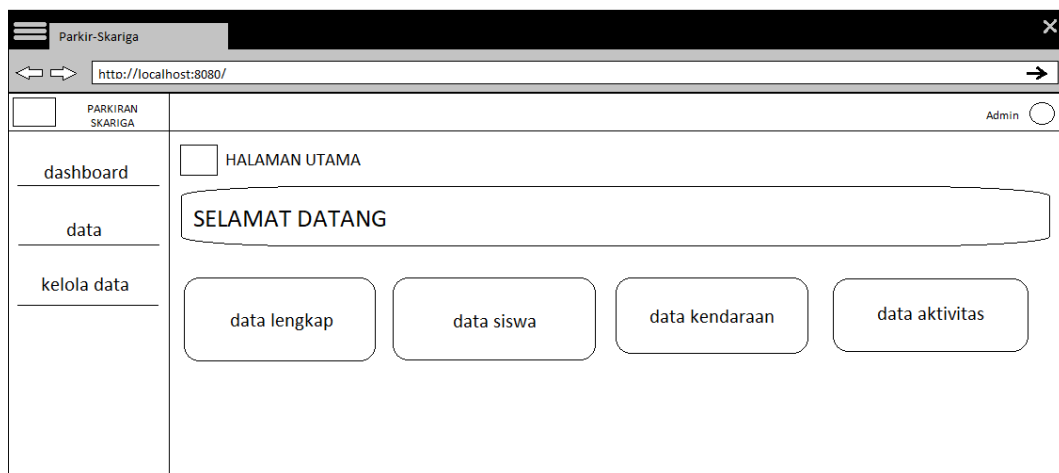
The screenshot shows a web browser window with the title 'Parkir-Skariga'. The address bar displays 'http://localhost:8080/'. The main content area features a login form with the following elements:

- A heading: 'Welcome Back!'
- An input field labeled 'username'.
- An input field labeled 'password'.
- A button labeled 'submit'.

Gambar 3.13 *Login*

D. Dashboard Admin

Desain halaman *dashboard* admin, berisi button yang dapat langsung mengakses data dan kelola data yang meliputi data lengkap, data siswa, data kendaraan, dan data aktivitas.



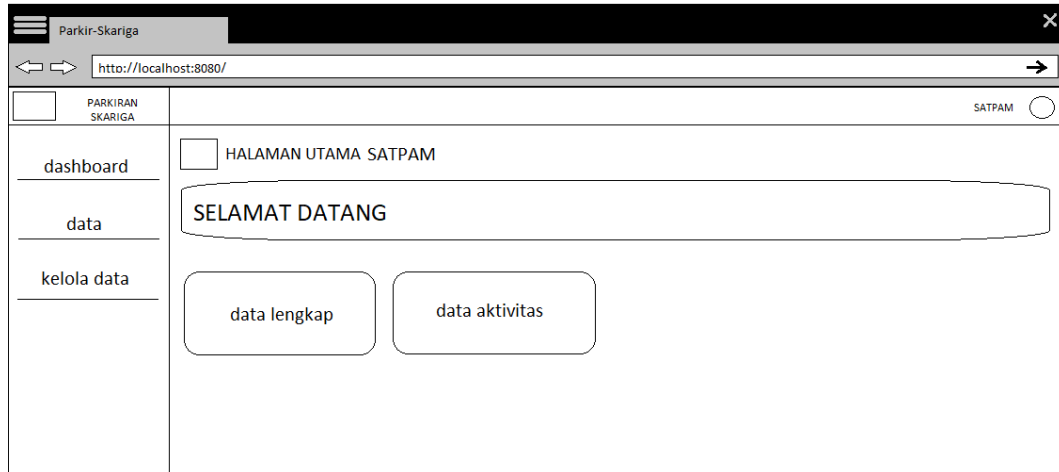
The screenshot shows an admin dashboard with the following layout:

- Browser:** Title 'Parkir-Skariga', Address 'http://localhost:8080/'.
- Sidebar (Left):** Contains a logo and three menu items: 'dashboard', 'data', and 'kelola data'.
- Main Content Area:**
 - Header: 'HALAMAN UTAMA'.
 - Greeting: A large box containing 'SELAMAT DATANG'.
 - Buttons: Four buttons arranged horizontally: 'data lengkap', 'data siswa', 'data kendaraan', and 'data aktivitas'.
- Top Right:** A user profile indicator showing 'Admin' and a circular icon.

Gambar 3.14 *Dashboard Admin*

E. Dashboard Satpam

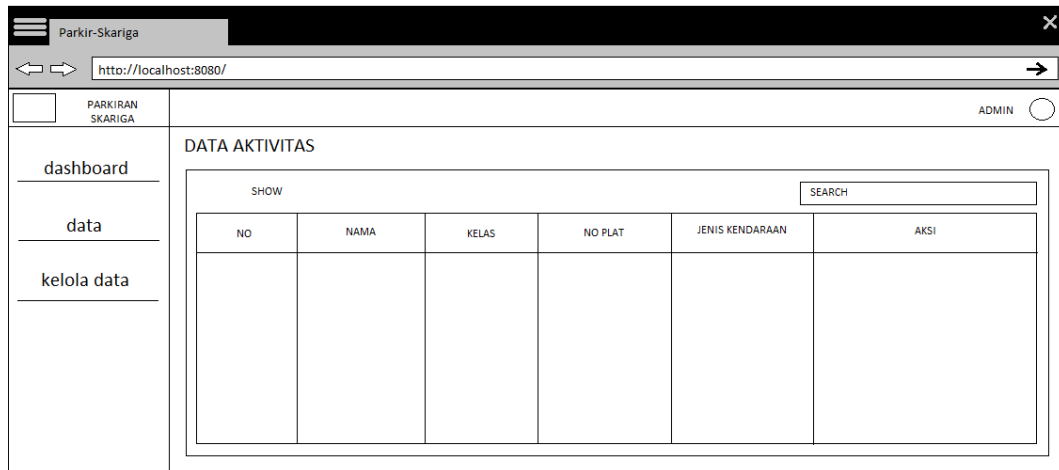
Desain halaman *dashboard* satpam, berisi button yang dapat langsung mengakses data dan kelola data yang meliputi data lengkap dan data aktivitas.



Gambar 3.15 Dashboard Satpam

F. Data Aktivitas (Admin)

Desain Halaman data aktivitas berisi tabel dengan kolom no, nama, kelas, no plat, jenis kelamin, dan aksi yang berisi button detail.



Gambar 3.16 Aktivitas (Admin)

G. Data Aktivitas (Satpam)

Desain Halaman data aktivitas berisi tabel dengan kolom no, nama, kelas, no plat, jenis kelamin, dan aksi yang berisi button detail.

The screenshot displays the 'DATA AKTIVITAS' page. On the left is a sidebar with a menu containing 'dashboard', 'data', and 'kelola data'. The main content area is titled 'DATA AKTIVITAS' and features a table with the following columns: NO, NAMA, KELAS, NO PLAT, JENIS KENDARAAN, and AKSI. Above the table, there is a 'SHOW' button and a search bar labeled 'SEARCH'. The top of the page has a header with 'PARKIRAN SKARIGA' on the left and 'SATPAM' on the right.

Gambar 3.17 Aktivitas (Satpam)

H. Detail Aktivitas

Desain tampilan detail data yang berisi data lengkap dari siswa, kendaraan, hingga aktivitas masuk dan keluar.

The screenshot displays the 'Detail Data' page. On the left is a sidebar with a menu containing 'dashboard', 'data', and 'kelola data'. The main content area is titled 'Detail Data' and features a form with the following fields: 'nama', 'kelas', 'no plat', 'tgl masuk', and 'tgl keluar'. Below the form is a 'back' button. The top of the page has a header with 'PARKIRAN SKARIGA' on the left and 'user' on the right.

Gambar 3.18 Detail Aktivitas

I. Kelola Siswa

Desain Halaman kelola siswa berisi tabel dengan kolom no, nis, nama, kelas, jurusan, dan aksi yang berisi button hapus dan ubah.

The screenshot shows a web application interface for managing students. The sidebar on the left contains links to 'dashboard', 'data', and 'kelola data'. The main content area is titled 'DATA SISWA' and features a 'tambah' button. Below this is a 'SHOW' section with a search bar and a table. The table has columns: NO, NIS, NAMA, KELAS, JURUSAN, and AKSI. The AKSI column contains 'UBAH' and 'HAPUS' buttons. The top right of the page shows 'ADMIN' and a profile icon.

Gambar 3.19 Kelola Siswa

J. Tambah Data Siswa

Desain Form Tambah Data Siswa berisi form input untuk menambah data siswa yaitu nama, kelas, nis, jurusan, dan guru wali.

The screenshot shows a web application interface for adding student data. The sidebar on the left contains links to 'dashboard', 'data', and 'kelola data'. The main content area is titled 'TAMBAH SISWA' and contains a form with input fields for NAMA, KELAS, NIS, JURUSAN, and GURU WALI. A 'SUBMIT' button is at the bottom. The top right of the page shows 'SATPAM' and a profile icon.

Gambar 3.20 Tambah Siswa

K. Ubah Data Siswa

Desain Form Ubah Data Siswa berisi form input untuk mengubah data siswa yaitu nama, kelas, nis, jurusan, dan guru wali.

The screenshot shows a web application interface for editing student data. The main content area is titled "EDIT DATA SISWA" and contains five input fields for "NAMA", "KELAS", "NIS", "JURUSAN", and "GURU WALI". Below these fields is a button labeled "UPDATE". The interface also features a sidebar on the left with links to "dashboard", "data", and "kelola data", and a top navigation bar with the text "PARKIRAN SKARIGA" and "SATPAM".

Gambar 3.21 Ubah Siswa

L. Kelola Kendaraan

Desain Halaman kelola kendaraan berisi tabel dengan kolom no, nis, no plat, jenis kendaraan, dan aksi yang berisi button hapus dan ubah.

The screenshot displays a web application interface for managing vehicles. The main content area is titled "DATA KENDARAAN" and includes a "tambah" button. Below this is a table with the following columns: "NO", "NIS", "NO PLAT", "JENIS KENDARAAN", and "AKSI". The "AKSI" column contains buttons for "UBAH" and "HAPUS". Above the table is a "SEARCH" input field and a "SHOW" button. The interface also features a sidebar on the left with links to "dashboard", "data", and "kelola data", and a top navigation bar with the text "PARKIRAN SKARIGA" and "ADMIN".

Gambar 3.22 Kelola Kendaraan

M. Tambah Data Kendaraan

Desain Form Tambah Data Kendaraan berisi form input untuk menambah data kendaraan yaitu nis, no plat, dan jenis kendaraan.

The screenshot shows a web browser window with the URL `http://localhost:8080/`. The page title is 'Parkir-Skariga'. On the left, there is a sidebar menu with the following items: 'dashboard', 'data', and 'kelola data'. The main content area displays a form titled 'TAMBAH DATA KENDARAAN'. The form contains three input fields: 'NIS', 'NO PLAT', and 'JENIS KENDARAAN'. Below these fields is a 'SUBMIT' button. The top right corner of the page shows the user 'SATPAM' and a profile icon.

Gambar 3.23 Tambah Kendaraan

N. Ubah Data Kendaraan

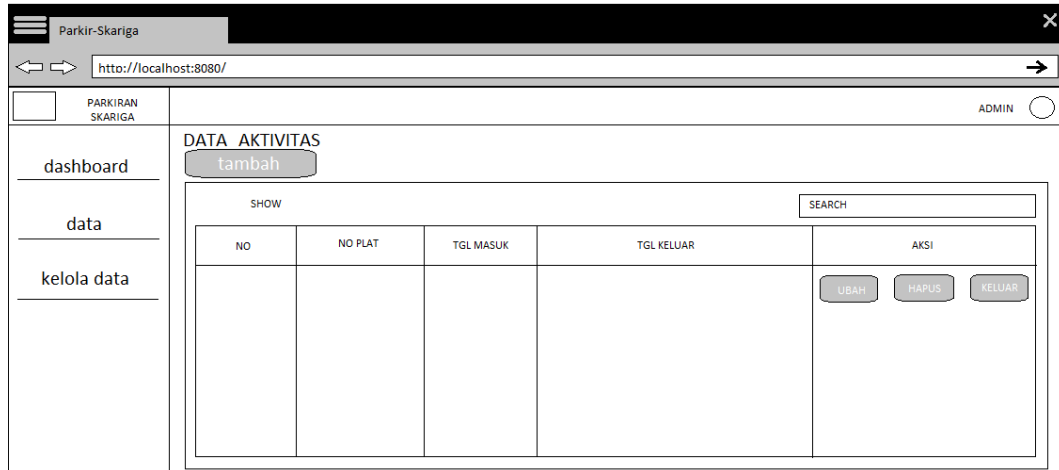
Desain Form Ubah Data Kendaraan berisi form input untuk mengubah data kendaraan yaitu nis, no plat, dan jenis kendaraan.

The screenshot shows the same web browser window as before, but the main content area displays a form titled 'EDIT DATA KENDARAAN'. This form also contains three input fields: 'NIS', 'NO PLAT', and 'JENIS KENDARAAN'. Below these fields is an 'UPDATE' button. The sidebar menu and the top navigation bar remain the same as in the previous screenshot.

Gambar 3.24 Ubah Kendaraan

O. Kelola Aktivitas (Admin)

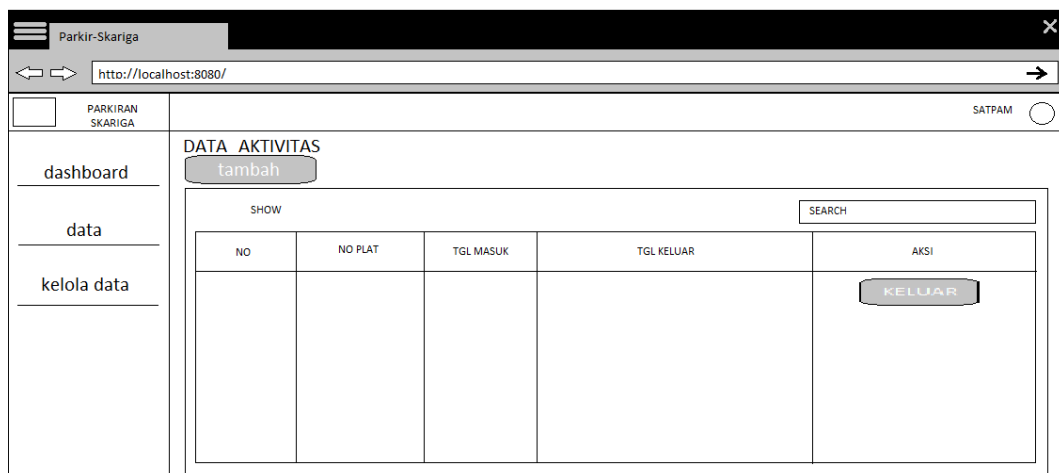
Desain Halaman kelola aktivitas berisi tabel dengan kolom no, no plat, tanggal masuk, tanggal keluar, dan aksi yang berisi button hapus, ubah dan keluar.



Gambar 3.25 Kelola Aktivitas (Admin)

P. Kelola Aktivitas (Satpam)

Desain Halaman kelola aktivitas berisi tabel dengan kolom no, no plat, tanggal masuk, tanggal keluar, dan aksi yang berisi button keluar.



Gambar 3.26 Kelola Aktivitas (Satpam)

Q. Tambah Data Aktivitas

Desain Form Tambah Data Aktivitas berisi form input untuk menambah data aktivitas yaitu no plat.

The screenshot shows a web browser window with the title 'Parkir-Skariga'. The address bar displays 'http://localhost:8080/'. The page has a sidebar menu on the left with a logo and the text 'PARKIRAN SKARIGA', and three menu items: 'dashboard', 'data', and 'kelola data'. The main content area features a form titled 'TAMBAH DATA AKTIVITAS'. This form contains a single text input field labeled 'NO PLAT' and a 'SUBMIT' button below it. In the top right corner of the main area, there is a 'SATPAM' label and a circular profile icon.

Gambar 3.27 Tambah Aktivitas

R. Ubah Data Aktivitas

Desain Form Ubah Data Aktivitas berisi form input untuk mengubah data aktivitas yaitu no plat dan tanggal masuk.

The screenshot shows the same web browser window as before, but the main content area displays a form titled 'EDIT DATA AKTIVITAS'. This form contains two text input fields: 'NO PLAT' and 'TGL MASUK', with an 'UPDATE' button positioned below them. The sidebar menu and the 'SATPAM' profile icon remain the same as in the previous image.

Gambar 3.28 Ubah Aktivitas

S. Logout

Desain halaman *logout* berisi *pop up* konfirmasi apakah pengguna yakin untuk keluar aplikasi atau tidak.



Gambar 3.29 Logout

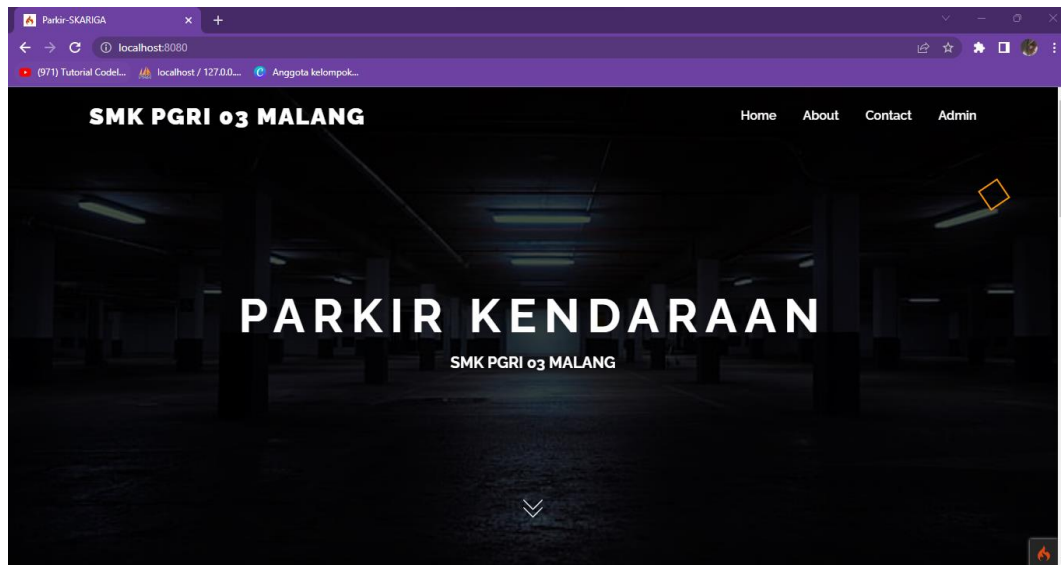
BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Pembuatan Aplikasi Parkir

A. *Home*

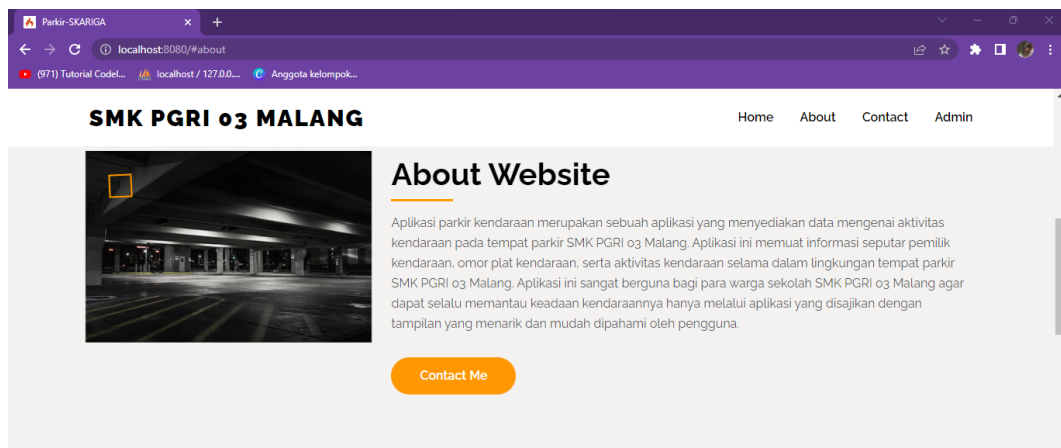
Hasil tampilan *Home* yang berisi judul aplikasi, dan beberapa fitur untuk mengakses halaman lainnya. Pada tengah bawah juga terdapat panah yang bisa ditekan dan akan turun ke halaman *about*.



Gambar 4.1 Halaman *Home*

B. *About*

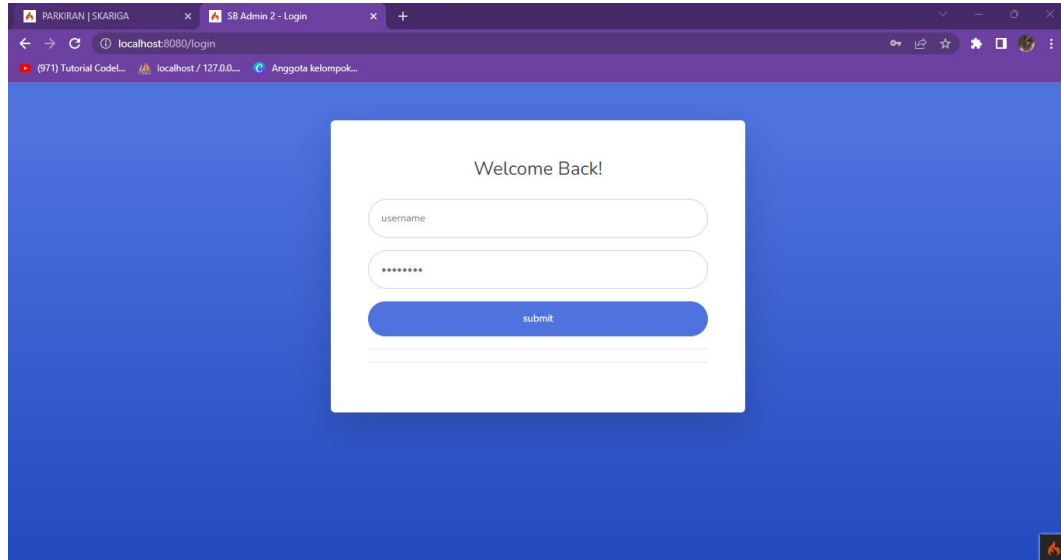
Hasil halaman *About* berisi tentang penjelasan mengenai cara kerja aplikasi yang dibuat serta terdapat button untuk menuju ke halaman *home*.



Gambar 4.2 Halaman *About*

C. Login

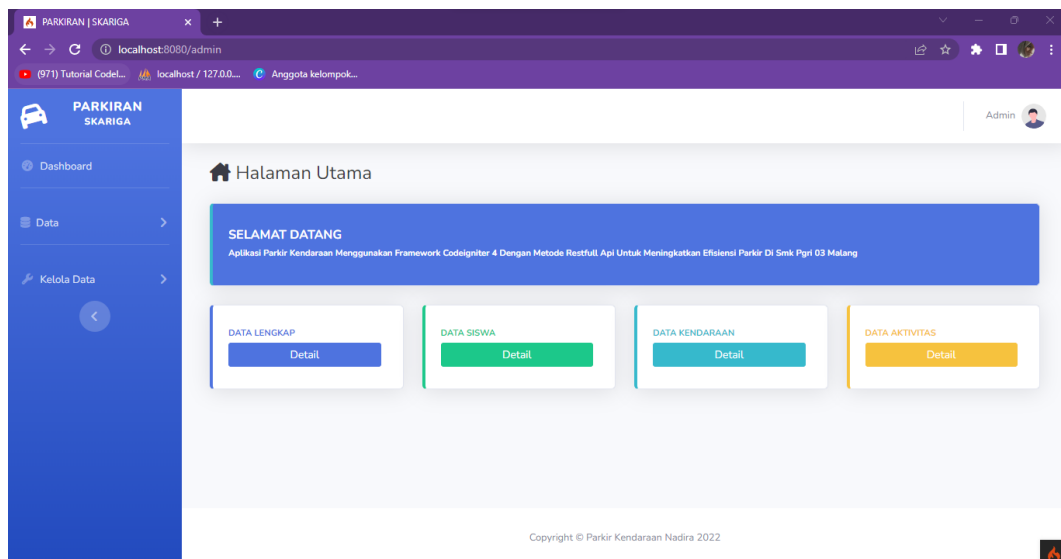
Hasil form *login* berisi inputan untuk admin dan satpam yang berisi *username* dan *password*.



Gambar 4.3 Halaman *Login*

D. Dashboard Admin

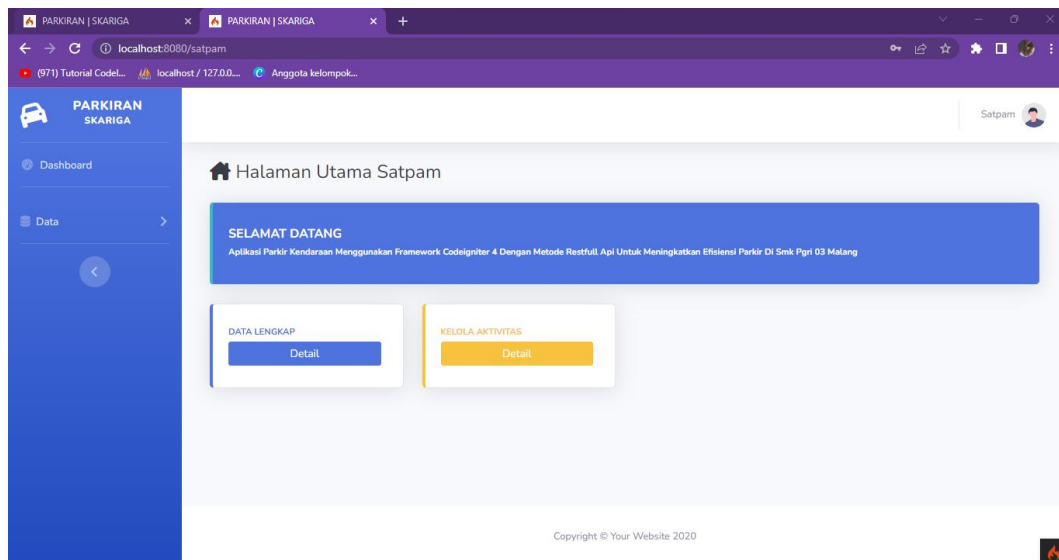
Hasil halaman *dashboard* admin, berisi button yang dapat langsung mengakses data dan kelola data yang meliputi data lengkap, data siswa, data kendaraan, dan data aktivitas.



Gambar 4.4 Halaman *Dashboard* Admin

E. Dashboard Satpam

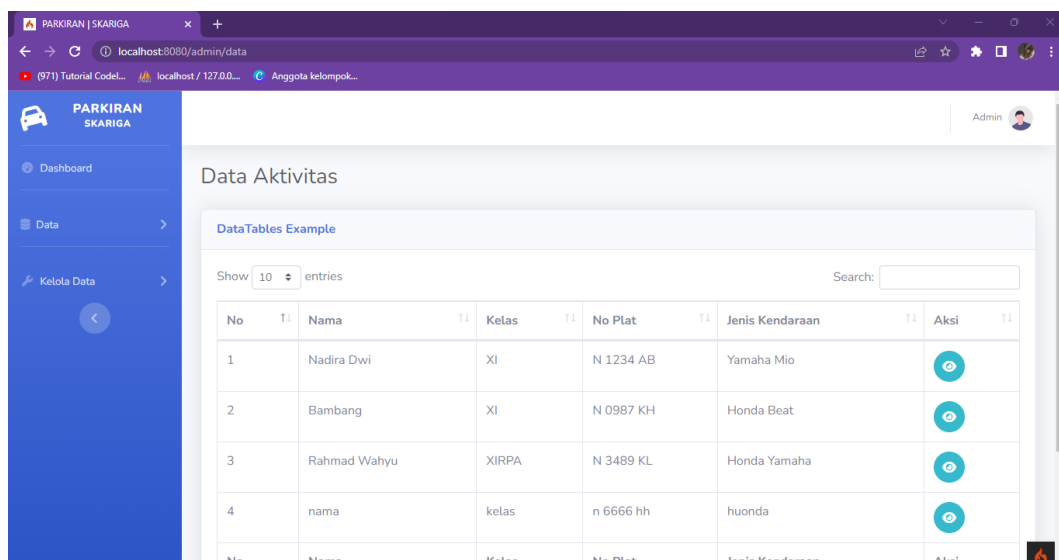
Hasil halaman *dashboard* satpam, berisi button yang dapat langsung mengakses data dan kelola data yang meliputi data lengkap dan data aktivitas.



Gambar 4.5 Halaman *Dashboard* Satpam

F. Aktivitas (Admin)

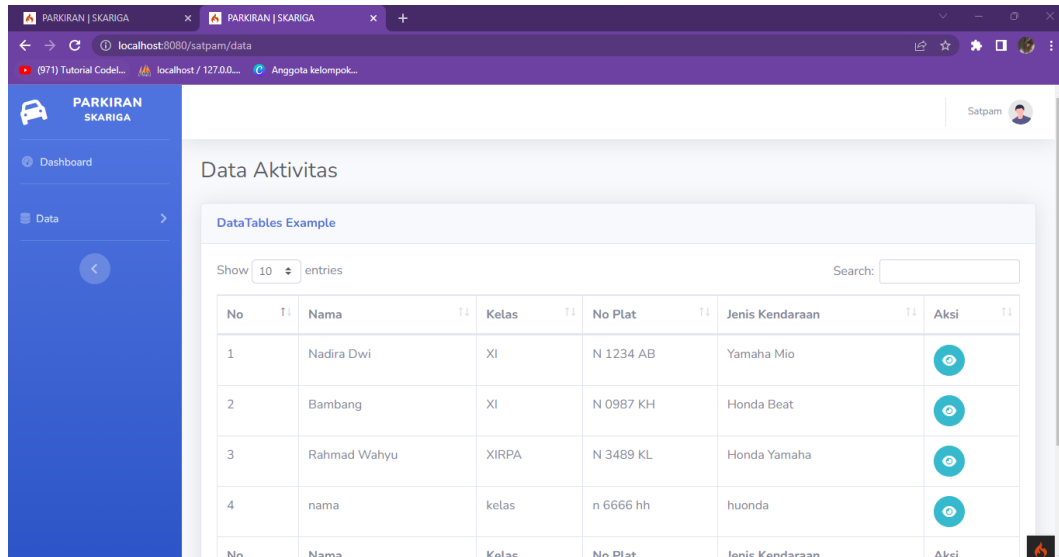
Hasil Halaman data aktivitas berisi tabel dengan kolom no, nama, kelas, no plat, jenis kelamin, dan aksi yang berisi button detail.



Gambar 4.6 Halaman Data Aktivitas

G. Aktivitas (Satpam)

Hasil Halaman data aktivitas berisi tabel dengan kolom no, nama, kelas, no plat, jenis kelamin, dan aksi yang berisi button detail.

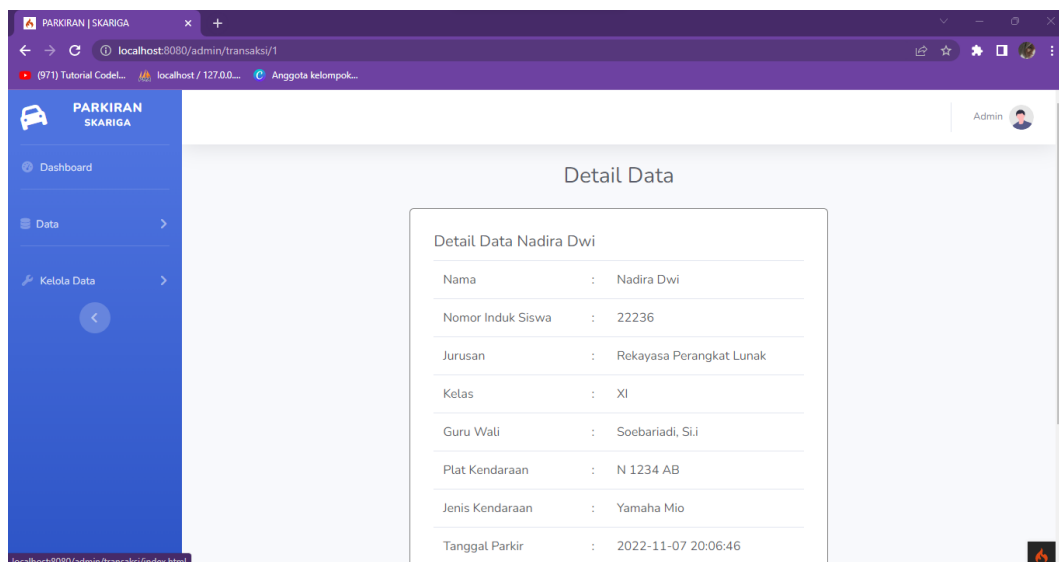


No	Nama	Kelas	No Plat	Jenis Kendaraan	Aksi
1	Nadira Dwi	XI	N 1234 AB	Yamaha Mio	
2	Bambang	XI	N 0987 KH	Honda Beat	
3	Rahmad Wahyu	XIRPA	N 3489 KL	Honda Yamaha	
4	nama	kelas	n 6666 hh	huonda	
No	Nama	Kelas	No Plat	Jenis Kendaraan	Aksi

Gambar 4.7 Halaman Data Aktivitas

H. Detail Aktivitas

Hasil tampilan detail data yang berisi data lengkap dari siswa, kendaraan, hingga aktivitas masuk dan keluar.

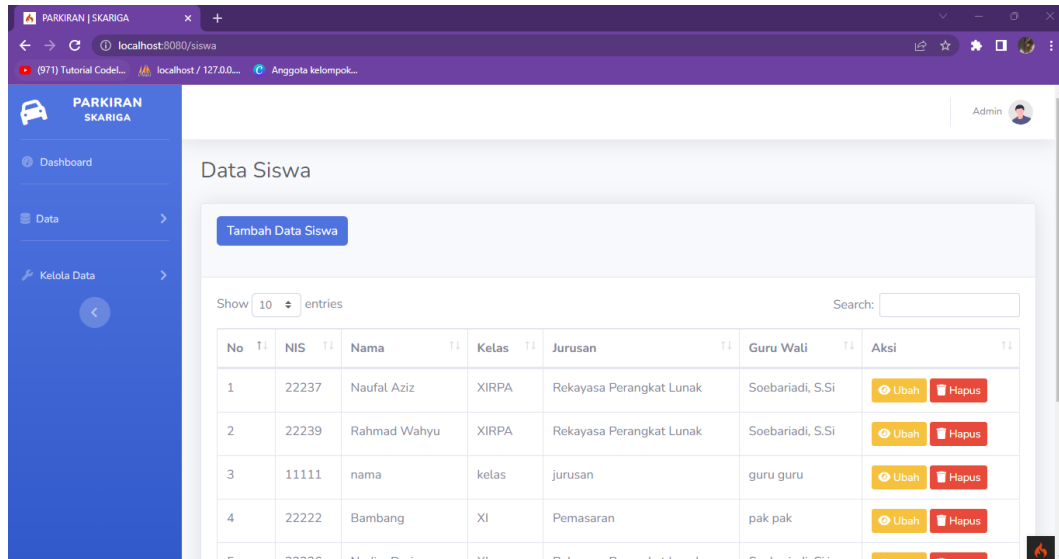


Detail Data Nadira Dwi	
Nama	Nadira Dwi
Nomor Induk Siswa	22236
Jurusan	Rekayasa Perangkat Lunak
Kelas	XI
Guru Wali	Soebariadi, Si.i
Plat Kendaraan	N 1234 AB
Jenis Kendaraan	Yamaha Mio
Tanggal Parkir	2022-11-07 20:06:46

Gambar 4.8 Halaman Detail Data

I. Kelola Siswa

Hasil Halaman kelola siswa berisi tabel dengan kolom no, nis, nama, kelas, jurusan, dan aksi yang berisi button hapus dan ubah.



PARKIRAN SKARIGA

Admin

Dashboard

Data

Kelola Data

Tambah Data Siswa

Show 10 entries

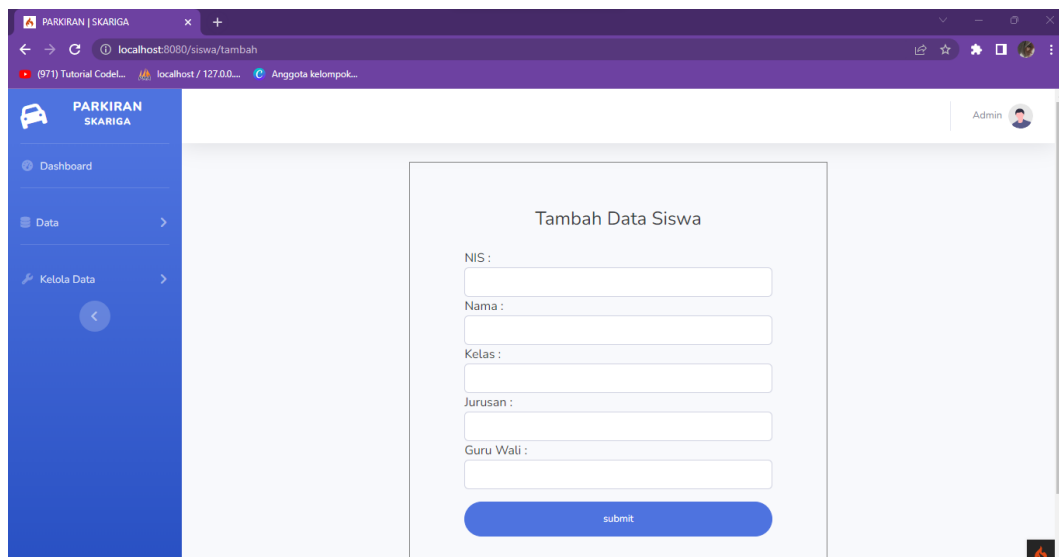
Search:

No	NIS	Nama	Kelas	Jurusan	Guru Wali	Aksi
1	22237	Naufal Aziz	XIRPA	Rekayasa Perangkat Lunak	Soebariadi, S.Si	Ubah Hapus
2	22239	Rahmad Wahyu	XIRPA	Rekayasa Perangkat Lunak	Soebariadi, S.Si	Ubah Hapus
3	11111	nama	kelas	jurusan	guru guru	Ubah Hapus
4	22222	Bambang	XI	Pemasaran	pak pak	Ubah Hapus
5	22236	Narlira Dwi	XI	Rekayasa Perangkat Lunak	Soebariadi, S.Si	Ubah Hapus

Gambar 4.9 Halaman Kelola Siswa

J. Tambah Siswa

Hasil Form Tambah Data Siswa berisi form input untuk menambah data siswa yaitu nama, kelas, nis, jurusan, dan guru wali.



PARKIRAN SKARIGA

Admin

Dashboard

Data

Kelola Data

Tambah Data Siswa

NIS :

Nama :

Kelas :

Jurusan :

Guru Wali :

submit

Gambar 4.10 Halaman Tambah Siswa

K. Update Siswa

Hasil Form Ubah Data Siswa berisi form input untuk mengubah data siswa yaitu nama, kelas, nis, jurusan, dan guru wali.

PARKIRAN SKARIGA

Admin

Dashboard

Data

Kelola Data

Edit Data Siswa

NIS :
22237

Nama :
Naufal Aziz

Kelas :
XIIRPA

Jurusan :
Rekayasa Perangkat Lunak

Guru Wali :
Soebariadi, S.Si

Update

Gambar 4.11 Halaman *Update* Siswa

L. Kelola Kendaraan

Hasil Halaman kelola kendaraan berisi tabel dengan kolom no, nis, no plat, jenis kendaraan, dan aksi yang berisi button hapus dan ubah.

PARKIRAN SKARIGA

Admin

Dashboard

Data

Kelola Data

Data Kendaraan

Tambah Data Kendaraan

Show 10 entries

Search:

No	NIS	Nomor Plat	Jenis Kendaraan	Aksi
1	22236	N 1234 AB	Yamaha Mio	Ubah Hapus
2	22222	N 0987 KH	Honda Beat	Ubah Hapus
3	22239	N 3489 KL	Honda Yamaha	Ubah Hapus
4	11111	n 6666 hh	huonda	Ubah Hapus
5	22222	n 1234 ab	...	Ubah Hapus

Gambar 4.12 Halaman Kelola Kendaraan

M. Tambah Kendaraan

Hasil Form Tambah Data Kendaraan berisi form input untuk menambah data kendaraan yaitu nis, no plat, dan jenis kendaraan.

Gambar 4.13 Halaman Tambah Kendaraan

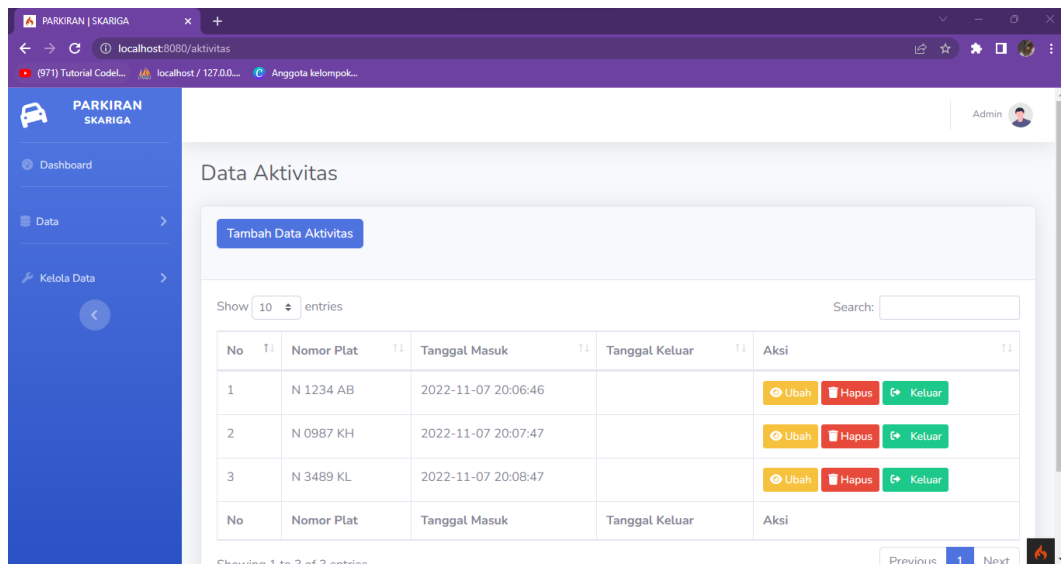
N. *Update* Kendaraan

Hasil Form Ubah Data Kendaraan berisi form input untuk mengubah data kendaraan yaitu nis, no plat, dan jenis kendaraan.

Gambar 4.14 Halaman *Update* Kendaraan

O. Kelola Aktivitas (Admin)

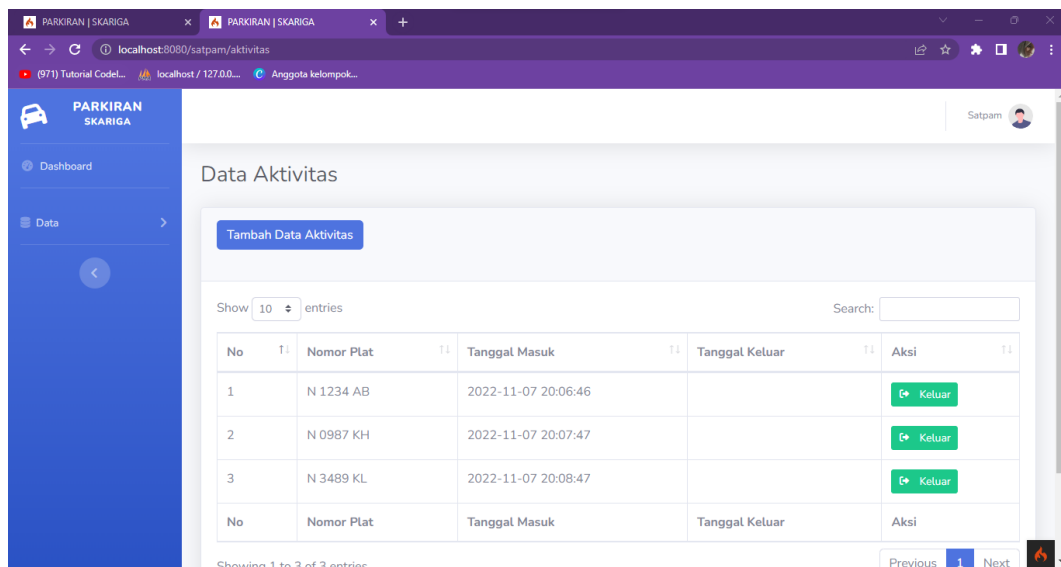
Hasil Halaman kelola aktivitas berisi tabel dengan kolom no, no plat, tanggal masuk, tanggal keluar, dan aksi yang berisi button hapus, ubah dan keluar.



Gambar 4.15 Halaman Kelola Aktivitas (Admin)

P. Kelola Aktivitas (Satpam)

Hasil Halaman kelola aktivitas berisi tabel dengan kolom no, no plat, tanggal masuk, tanggal keluar, dan aksi yang berisi button keluar.



Gambar 4.16 Halaman Kelola Aktivitas (Satpam)

Q. Tambah Aktivitas

Hasil Form Tambah Data Aktivitas berisi form input untuk menambah data aktivitas yaitu no plat.

The screenshot shows a web browser window with the URL `localhost:8080/aktivitas/tambah`. The application has a blue sidebar with the logo 'PARKIRAN SKARIGA' and navigation links: 'Dashboard', 'Data', and 'Kelola Data'. The main content area is titled 'Tambah Data Aktivitas' and contains a form with a single input field labeled 'Nomor Plat :'. Below the input field is a blue 'submit' button. The footer of the page reads 'Copyright © Your Website 2020'.

Gambar 4.17 Halaman Tambah Aktivitas

R. *Update* Aktivitas

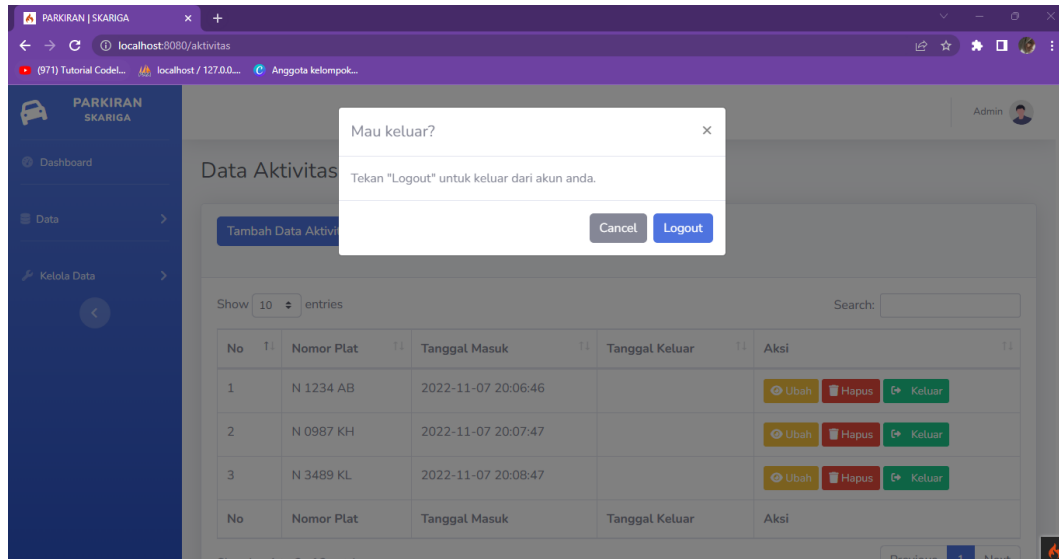
Hasil Form Ubah Data Aktivitas berisi form input untuk mengubah data aktivitas yaitu no plat dan tanggal masuk.

The screenshot shows a web browser window with the URL `localhost:8080/aktivitas/edit/1`. The application has a blue sidebar with the logo 'PARKIRAN SKARIGA' and navigation links: 'Dashboard', 'Data', and 'Kelola Data'. The main content area is titled 'Edit Data Aktivitas' and contains a form with two input fields: 'Nomor Plat :' (containing 'N 1234 AB') and 'Tanggal Masuk :' (containing '2022-11-07 20:06:46'). Below the input fields is a blue 'Update' button. The footer of the page reads 'Copyright © Parkir Kendaraan Nadira 2022'.

Gambar 4.18 Halaman *Update* Aktivitas

S. Logout

Hasil halaman *logout* berisi pop up konfirmasi apakah pengguna yakin untuk keluar aplikasi atau tidak.



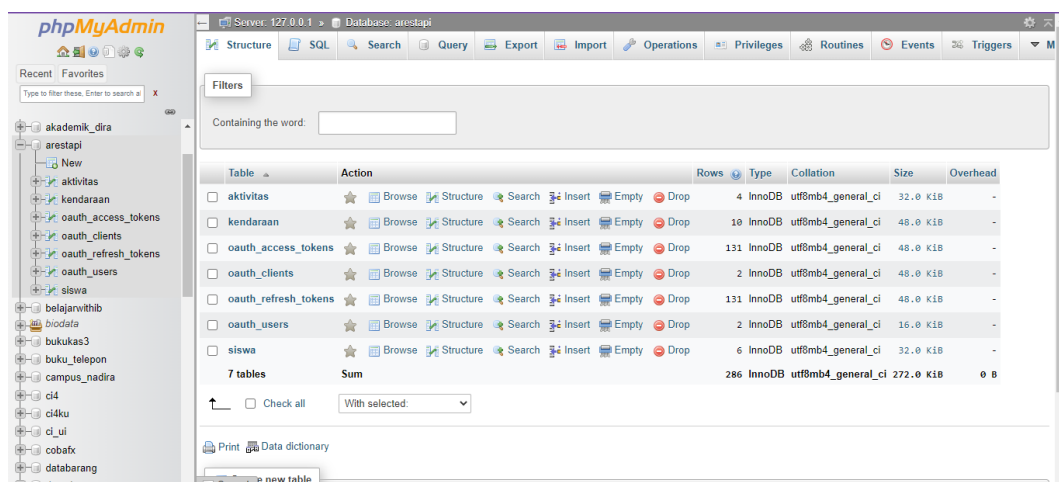
Gambar 4.19 Halaman *Logout*

4.2 Hasil Pembuatan Sistem *Database*

Berikut merupakan hasil dari pembuatan *database* aplikasi parkir kendaraan

4.2.1 Pembuatan *Database*

Hasil Pembuatan *Database* ini terdiri dari 7 tabel, yaitu *table aktivitas*, *table kendaraan*, *table oauth_access_token*, *table oauth clients*, *table oauth refresh token*, *table oauth user*, dan *table siswa*.



Gambar 4.20 *Database*

4.2.2 Pembuatan Tabel

Tabel yang digunakan pada Aplikasi Parkir Kendaraan ini :

A. Table Siswa

Dalam *table* siswa terdapat 6 kolom yaitu, *id_siswa* sebagai primary key , *nis*, *nama_siswa*, *kelas*, *jurusan*, dan *guru_wali*.

		<i>id_siswa</i>	<i>nis</i>	<i>nama_siswa</i>	<i>kelas</i>	<i>jurusan</i>	<i>guru_wali</i>
☐	Edit Copy Delete	3	22237	Naufal Aziz	XIRPA	Rekayasa Perangkat Lunak	Soebariadi, S.Si
☐	Edit Copy Delete	4	22239	Rahmad Wahyu	XIRPA	Rekayasa Perangkat Lunak	Soebariadi, S.Si
☐	Edit Copy Delete	8	11111	nama	kelas	jurusan	guru guru
☐	Edit Copy Delete	14	22222	Bambang	XI	Pemasaran	pak pak
☐	Edit Copy Delete	17	22236	Nadira Dwi	XI	Rekayasa Perangkat Lunak	Soebariadi, S.Si
☐	Edit Copy Delete	19	22233	Surya	III	Kimia Listrik	Soebariadi, S.Si

Gambar 4.21 Tabel Siswa

B. Table Kendaraan

Pada *table* kendaraan terdapat 4 kolom yaitu, *id_kendaraan* sebagai primary key, *no_plat*, *jenis_kendaraan*, dan *nis*.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 <i>id_kendaraan</i>	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 <i>no_plat</i>	varchar(10)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 <i>jenis_kendaraan</i>	varchar(255)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 <i>nis</i>	int(10)			No	None			Change Drop More

Gambar 4.22 Tabel Kendaraan

C. Table Aktivitas

Table aktivitas terdapat 4 kolom yaitu, *id_aktivitas* sebagai primary key, *no_plat*, *tgl_masuk*, dan *tgl_keluar*.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 <i>id_aktivitas</i>	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 <i>no_plat</i>	varchar(10)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 <i>tgl_masuk</i>	datetime			Yes	current_timestamp()			Change Drop More
☐	4 <i>tgl_keluar</i>	datetime			Yes	NULL			Change Drop More

Gambar 4.23 Tabel Aktivitas

D. Oauth Users

Pada *table oauth users* terdapat 7 kolom yaitu, *username* sebagai primary key, *password*, *first_name*, *last_name*, *email*, *email_verified*, dan *scope*.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 username	varchar(80)	utf8mb4_general_ci		No	None			Change Drop More
☐	2 password	varchar(80)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	3 first_name	varchar(80)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	4 last_name	varchar(80)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	5 email	varchar(80)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	6 email_verified	tinyint(1)			Yes	NULL			Change Drop More
☐	7 scope	varchar(4000)	utf8mb4_general_ci		Yes	NULL			Change Drop More

Gambar 4.24 Tabel *Oauth User*

E. Oauth Clients

Tabel *oauth clients* terdiri dari 6 kolom yaitu, *client_id* sebagai primary key, *client_secret*, *redirect_uri*, *grant_types*, *scope*, dan *user_id*.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 client_id	varchar(80)	utf8mb4_general_ci		No	None			Change Drop More
☐	2 client_secret	varchar(80)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	3 redirect_uri	varchar(2000)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	4 grant_types	varchar(80)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	5 scope	varchar(4000)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	6 user_id	varchar(80)	utf8mb4_general_ci		Yes	NULL			Change Drop More

Gambar 4.25 Tabel *Oauth Client*

F. *Oauth Access Token*

Table *oauth access token* terdapat 5 kolom yaitu, *access_token* sebagai primary key, *client_id*, *user_id*, *expires*, dan *scope*.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 access_token 🔑	varchar(40)	utf8mb4_general_ci		No	None			Change
☐	2 client_id 🔑	varchar(80)	utf8mb4_general_ci		No	None			Change
☐	3 user_id 🔑	varchar(80)	utf8mb4_general_ci		Yes	NULL			Change
☐	4 expires	timestamp			No	current_timestamp()	ON UPDATE CURRENT_TIMESTAMP()		Change
☐	5 scope	varchar(4000)	utf8mb4_general_ci		Yes	NULL			Change

Gambar 4.26 Tabel Access Token

G. *Oauth Refresh Token*

Table *oauth refresh token* memiliki 5 kolom di dalamnya yaitu, *refresh_token* sebagai primary key, *client_id*, *user_id*, *expires*, dan *scope*.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 refresh_token 🔑	varchar(40)	utf8mb4_general_ci		No	None			Change
☐	2 client_id 🔑	varchar(80)	utf8mb4_general_ci		No	None			Change
☐	3 user_id 🔑	varchar(80)	utf8mb4_general_ci		Yes	NULL			Change
☐	4 expires	timestamp			No	current_timestamp()	ON UPDATE CURRENT_TIMESTAMP()		Change
☐	5 scope	varchar(4000)	utf8mb4_general_ci		Yes	NULL			Change

Gambar 4.27 Tabel Refresh Token

4.2.3 Pembuatan Script

Berikut ini adalah contoh script yang digunakan pada aplikasi parkir kendaraan ini :

A. Show Data

Berikut adalah script untuk menampilkan data data yang telah dibuat di *database* dan dipanggil pada file model.

```
public function index()
{
    $siswaModel = new SiswaModel();
    $kendaraanModel = new KendaraanModel();
    $aktivitasModel = new AktivitasModel();

    $data = array(
        'dataSiswa' => $siswaModel->findAll(),
        'dataKendaraan' => $kendaraanModel->findAll(),
        'dataAktivitas' => $aktivitasModel->findAll(),
    );

    return $this->respond($data);
}
```

Gambar 4.28 Show Data

B. Create Data

Berikut adalah script untuk menambah data siswa.

```
public function create()
{
    $kendaraanModel = new KendaraanModel();

    helper(['form']);

    $rules = [
        'no_plat' => 'required|max_length[10]',
        'jenis_kendaraan' => 'required|max_length[30]',
        'nis' => 'required|max_length[10]'
    ];

    if (!$this->validate($rules)) {
        return $this->fail($this->validator->getErrors());
    } else {
        $data = [
            'no_plat' => $this->request->getVar('no_plat'),
            'jenis_kendaraan' => $this->request->getVar('jenis_kendaraan'),
            'nis' => $this->request->getVar('nis'),
        ];

        if ($data) {
            if ($this->checkNis($data['nis'])) {
                if ($this->checkPlat($data['no_plat'])) {
                    return $this->respondCreated(array("Message" => "plat sudah ada", "Code" => "401"));
                } else {
                    if ($kendaraanModel->insert($data)) {
                        $data['Code'] = "201";
                        return $this->respondCreated($data);
                    }
                }
            } else {
                return $this->respondCreated(array("Message" => "data nis tidak ditemukan", "Code" => "402"));
            }
        }
    }
}
```

Gambar 4.29 Create Data

C. Update Data

Berikut adalah script untuk mengedit data pada tabel kendaraan.

```
public function update($id = null)
{
    $kendaraanModel = new KendaraanModel();
    helper(['form', 'array']);

    $rules = [
        'no_plat' => 'required|max_length[10]',
        'jenis_kendaraan' => 'required|max_length[30]',
        'nis' => 'required|max_length[10]'
    ];

    if (!$this->validate($rules)) {
        return $this->fail($this->validator->getErrors());
    } else {
        $input = $this->request->getRawInput();

        $data = [
            'id_kendaraan' => $id,
            'no_plat' => $input['no_plat'],
            'jenis_kendaraan' => $input['jenis_kendaraan'],
            'nis' => $input['nis'],
        ];

        $kendaraanModel->save($data);
        return $this->respond($data);
    }
}
```

Gambar 4.30 Update Data

D. Delete Data

Berikut adalah script untuk menghapus data pada tabel kendaraan.

```
public function delete($id = null)
{
    $kendaraanModel = new KendaraanModel();
    $data = $kendaraanModel->find($id);
    if ($data) {
        $kendaraanModel->delete($id);
        return $this->respondDeleted($data);
    } else {
        return $this->failNotFound('DATA NE GA ONOK WOIIII');
    }
}
```

Gambar 4.31 Delete Data

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Dengan proses pembuatan Aplikasi Parkir Kendaraan Menggunakan Framework Codeigniter 4 Dengan Metode Restfull Api Untuk Meningkatkan Efisiensi Parkir Di Smk Pgri 03 Malang ini maka dapat disimpulkan bahwa :

1. Cara membuat Aplikasi Parkir Kendaraan Menggunakan Framework Codeigniter 4 dengan Metode Restfull API untuk Meningkatkan Efisiensi Parkir di SMK PGRI 03 Malang adalah dengan membedakan file Codeigniter untuk server (yang mengakses *database*) dengan file Codeigniter untuk mendesain tampilan.
2. Cara mengoperasikan Aplikasi Parkir Kendaraan Menggunakan Framework Codeigniter 4 dengan Metode Restfull API untuk Meningkatkan Efisiensi Parkir di SMK PGRI 03 Malang adalah, satpam menginputkan nomor plat yang melakukan aktivitas masuk dan keluar parkiran. Nomor plat yang diinputkan hanya yang sudah di daftarkan oleh admin.

5.2 Saran

Penulis menyadari bahwa masih ada beberapa kekurangan di Aplikasi Parkir Kendaraan ini yaitu :

1. Aplikasi masih belum terdapat fitur tampilan untuk siswa, dikarenakan ketidak pandai-an penulis dalam mengatur waktu dalam pengerjaan project.
2. Tampilan pada tiap halaman masih kurang bervariasi, hanya terpaku dalam satu desain tampilan yang sama, jadi kelihatan membosankan.
3. Penggunaan bahasa harus difokuskan pada satu bahasa, atau dapat ditambah fitur mengubah bahasa Indonesia menjadi bahasa Inggris pada keseluruhan isi project.
4. Pada detail aktivitas bisa ditambah rekapitulasi aktivitas parkir secara keseluruhan.

DAFTAR PUSTAKA

- Litalia. - . *Pengertian Aplikasi, Fungsi dan Contoh Aplikasi Lengkap*. (Online) (https://www.jurnalponsel.com/pengertian-aplikasi/#Pengertian_Aplikasi_Menurut_Para_Ahli), diakses 16 Oktober 2022.
- Setiawan, Rony. 2021. *Apa itu Framework? Developer Wajib Tahu*. (Online) (<https://www.dicoding.com/blog/apa-itu-Framework/>), diakses 16 Agustus 2022.
- Anonim. - . *Pengertian CSS dan Fungsinya serta Cara Penulisan CSS*. (Online) (<https://www.temukanpengertian.com/2013/01/pengertian-css.html>), diakses 17 Agustus 2022.
- Ubaidah, Ahmad Nur. 2020. *Pengertian JavaScript, Keuntungan dan Kekurangan Menggunakannya*. (Online) ([https://www.logique.co.id/blog/2020/11/18/keuntungan-pengertian-javascript/#:~:text=Menurut%20Mac%20Bride%20\(2007\)%20dalam,aktivitas%20yang%20terjadi%20pada%20halaman](https://www.logique.co.id/blog/2020/11/18/keuntungan-pengertian-javascript/#:~:text=Menurut%20Mac%20Bride%20(2007)%20dalam,aktivitas%20yang%20terjadi%20pada%20halaman)), diakses 17 Agustus 2022.
- Litalia. - . *Pengertian PHP Beserta Fungsi Dan Sejarah PHP Dalam Pemograman*. (Online) (<https://www.jurnalponsel.com/pengertian-php/>), diakses 17 Agustus 2022.
- Ritonga, Pahmi. 2016. *Pengertian HTML Menurut Para Ahli dan Pakar*. (Online) (<https://bangpahmi.com/pengertian-html-menurut-para-ahli-dan-pakar/>), diakses 17 Agustus 2022.
- Admin, Media. 2022. *Pengertian Framework CodeIgniter Menurut Para Ahli*. (Online) (<https://creatormedia.my.id/pengertian-Framework-codeigniter-menurut-para-ahli/>), diakses 18 Agustus 2022.
- Saputra, Yoga Perdana. 2020. *Penjelasan Konsep MVC dalam Pemrograman Website*. (Online) (<https://inovatifku.com/konsep-mvc-dalam-Website/>), diakses 18 Agustus 2022.
- Anonim. - . *Apa itu API Restful?*. (Online) (<https://aws.amazon.com/id/what-is/restful-api/>), diakses 18 Agustus 2022.
- Mulachela, Husen. 2021. *Database Adalah: Pengertian dan Jenisnya*. (Online) (<https://katadata.co.id/intan/digital/61c04e3f62f5b/database-adalah-pengertian-dan-jenisnya>), diakses 29 Agustus 2022.
- Anonim. - . *DBMS Adalah: Pengertian, Fungsi, Jenis, dan Kelebihan Kekurangannya*. (Online) (<https://accurate.id/teknologi/dbms-adalah/>), diakses 29 Agustus 2022.

- Adani, Muhammad Robith. 2020. *Apa itu MySQL: Pengertian, Fungsi, beserta Kelebihan*. (Online) (<https://www.sekawanmedia.co.id/blog/pengertian-mysql/>), diakses 29 Agustus 2022.
- Sekarningrum, Anisa. 2021. *XAMPP adalah: Pengertian, fungsi, 5 komponen, dan cara menggunakannya*. (Online) (<https://www.ekrut.com/media/xampp-adalah>), diakses 29 Agustus 2022.
- Anonim. 2021. *Apa Itu Apache? Penjelasan Mengenai Apache Web Server*. (Online) (<https://gudangssl.id/blog/apa-itu-apache/>), diakses 29 Agustus 2022.
- Adani, Muhammad Robith. 2021. *Ketahui Fungsi dan Kelebihan dari Penggunaan CRUD dalam Pemrograman*. (Online) (<https://www.sekawanmedia.co.id/blog/pengertian-crud/>), diakses 30 Agustus 2022.
- Yonata, Jefri. 2022. *Apa itu Bootstrap? Fungsi, Kelebihan, dan Kekurangannya*. (Online) (<https://www.dewaweb.com/blog/apa-itu-bootstrap/>), diakses 5 September 2022.
- Anonim. - . *Entity Relationship Diagram: Pengertian, Fungsi, dan Cara Membuatnya*. (Online) (<https://accurate.id/marketing-manajemen/entity-relationship-diagram/>), diakses 5 September 2022.
- Adani, Muhammad Robith. 2021. *Ketahui Apa itu Data Flow Diagram (DFD) Beserta Jenis dan Fungsinya*. (Online) (<https://www.sekawanmedia.co.id/blog/dfd-adalah/>), diakses 5 September 2022.
- Setiawan, Rony. 2021. *Flowchart Adalah: Fungsi, Jenis, Simbol, dan Contohnya*. (Online) (<https://www.dicoding.com/blog/flowchart-adalah/>), diakses 6 September 2022.
- Ariffudin, Muhammad. 2022. *Mengenal Visual Code Studio dan Fitur-Fitur Pentingnya*. (Online) (<https://www.niagahoster.co.id/blog/visual-code-studio/>), diakses 6 September 2022.
- Fathurrahman. 2021. *APA ITU POSTMAN DAN BAGAIMANA MENGGUNAKAN POSTMAN UNTUK MENGUJI API ?*. (Online) (<https://www.mataWebsite.com/blog/apa-itu-postman-dan-bagaimana-menggunakan-postman-untuk-menguji-api>), diakses 6 September 2022.
- Azhar, Retiani. 2016. *Contoh Narasi Biodata Penulis*. (Online) (<https://ralfiyahazhaar.wordpress.com/2016/01/10/contoh-narasi-biodata-penulis/comment-page-1/>), diakses 11 September 2022.

Anonim. - . Narasi Biodata Penulis. (Online) (<https://sehataktif.com/narasi-biodata-penulis/>), diakses 11 September 2022.

BIODATA PENULIS

Nadira Dwi Setiawati yang biasa dipanggil dengan nama “Nadira” atau “Dira” merupakan nama dari penulis laporan project work ini. Lahir 17 tahun yang lalu, lebih tepatnya pada hari Rabu tanggal 21 September 2005, di Kota Batu, Jawa Timur dan saat ini bertempat tinggal di Kota Batu. Penulis merupakan anak ke 2 dari 3 bersaudara, dari pasangan Imam Supono dan Ise Royana. penulis pertama kali masuk pendidikan di Raudlatul Atfal (RA) 04 Cempaka Putih pada tahun 2010 dan tamat 2012. Kemudian penulis melanjutkan pendidikan di Madrasah Ibtidaiyah (MI) Bustanul Ulum dan tamat pada 2018. Setelah tamat pendidikan MI, penulis melanjutkan ke jenjang yang lebih tinggi lagi di Sekolah Menengah Pertama (SMP) Negeri 01 Batu dan lulus pada 2021. Berikutnya penulis meneruskan pendidikan di Sekolah Menengah Kejuruan (SMK) PGRI 03 Malang pada Bidang Keahlian Teknologi Informasi dan Komunikasi dan mengambil pilihan Program Keahlian Pengembangan Perangkat Lunak dan Gim dengan Konsentrasi Keahlian Rekayasa Perangkat Lunak. Penulis memiliki ketertarikan dengan olahraga bulutangkis namun penulis tidak memiliki lapangan. Selain itu, penulis juga sangat gemar bermain game dan juga membaca novel maupun komik. Berikut adalah email penulis yang dapat dihubungi ndwi4231@gmail.com. Kemudian dapat juga menghubungi penulis pada nomor handphone, 089630193693. Jika ingin menambahkan teman pada media sosial Instagram, penulis juga memiliki akun yang bernama @dwi_dira.